

Federated & Context-Aware Stress Detection System



SESSION: 2025-2026

A Major Project Report

Submitted to

Rajiv Gandhi Proudhyogiki Vishwavidyalaya, Bhopal

towards the partial fulfilment of the Degree of

Bachelor of Technology

in

Electronics and Telecommunication Engineering

Guided by:

Prof. L. D. Malviya
Prof. Neeraj Malviya

Submitted by:

Shubh Badonia
Shivansh Agrawal
Rohit Agrawal
Vinita Sonvane

Shri Govindram Seksaria Institute of Technology and Science, Indore (M.P.)

Department of Electronics and Telecommunication Engineering

April 2026

**SHRI GOVINDRAM SEKSARIA INSTITUTE OF TECHNOLOGY AND
SCIENCE, INDORE (M.P.)**



RECOMMENDATION

This major project report entitled, **“Federated and Context-Aware Stress Detection System”**, submitted to the Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal, by **Vinita Sonvane , Shivansh Agrawal , Shubh Badonia , Rohit Agrawal** during the academic year **2025-2026** as a partial fulfilment for the award of the degree of **Bachelor of Technology in Electronics and Telecommunication Engineering**, is a record of student’s own work carried out by them under our direct supervision, in the **Department of Electronics and Telecommunication Engineering, S.G.S.I.T.S. Indore (M.P.)**. The work contained in the project report is a satisfactory account of their project work and is recommended for the award of the degree.

Guide Name

Prof. L. D. Malviya

Co-guide Name

Prof. Neeraj Malviya

Prof. S. K. Jain

(Head of Department)

Electronics and Telecommunication Engineering

**SHRI GOVINDRAM SEKSARIA INSTITUTE OF TECHNOLOGY AND
SCIENCE, INDORE (M.P.)**



CERTIFICATE

This is to certify that the major project report entitled “**Federated and Context-Aware Stress Detection System**”, submitted to the Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal, by **Vinita Sonvane (0801EE221107)**, **Shivansh Agrawal (0801EC221091)**, **Shubh Badonia (0801EC221091)** and **Rohit Agrawal (0801EC221084)** during the academic year **2025-2026**, is a record of their own work and is accepted in the partial fulfilment for the award of the degree of **Bachelor of Technology in Electronics and Telecommunication Engineering**.

Internal Examiner:

External Examiner:

Date:

Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Prof. L. D. Malviya, for his continuous guidance, valuable suggestions, and constant encouragement throughout the development of this project. His expertise and support played a crucial role in the successful completion of this work. We are also deeply thankful to our co-guide, Prof. Neeraj Malviya, for his insightful feedback and timely assistance, which greatly contributed to improving the quality of this project. We extend our heartfelt thanks to the Head of the Department and all faculty members of the Electronics and Telecommunication Engineering Department at Shri Govindram Seksaria Institute of Technology and Science, Indore, for providing the necessary resources, support, and academic environment required for the completion of this project. We would also like to acknowledge the efforts and cooperation among our team members, whose collaboration and dedication made this project possible. Finally, we express our sincere gratitude to our families and friends for their constant support, motivation, and encouragement throughout the course of this project.

Shubh Badonia

Shivansh Agrawal

Rohit Agrawal

Vinita Sonvane

ABSTRACT

Stress has become a major global health concern due to its direct association with cardiovascular disorders, mental health issues, and reduced productivity. Conventional stress detection systems often misclassify physical activity as stress, as both conditions lead to similar physiological changes such as increased heart rate, body temperature, and motion. This limitation results in inaccurate monitoring and reduces the reliability of existing wearable systems. This project proposes a Federated and Context-Aware Stress Detection System that aims to accurately distinguish between psychological stress and physical activity using a hybrid approach combining Internet of Things (IoT), machine learning, and federated learning. The system utilizes an ESP32 microcontroller integrated with multiple physiological sensors, including but not limited to MAX30100/MAX30102 for heart rate and SpO₂ monitoring, MAX30205 for skin temperature measurement, and MPU6050 for motion tracking. These sensors continuously collect real-time multimodal physiological data. The WESAD (Wearable Stress and Affect Detection) dataset and first party data is used for training and validating machine learning models such as Random Forest, Support Vector Machine (SVM), Decision Tree, and Random Forest. The collected data undergoes preprocessing steps including noise filtering, segmentation, and feature extraction to improve classification accuracy. The trained model classifies user states into two categories: stress and physical activity. To enhance privacy and personalization, the system incorporates a Federated Learning framework, where model training is performed locally on edge devices, and only model parameters are shared with a central server using aggregation techniques such as Federated Averaging (FedAvg). This ensures that sensitive physiological data remains on the device while enabling collaborative model improvement. The proposed system provides an efficient, accurate, and privacy-preserving solution for real-time stress monitoring. It has potential applications in healthcare, fitness tracking, workplace monitoring, and personalized wellness systems.

Contents

List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
1. Introduction	
1.1 Necessity	1
1.2 Motivation	2
1.3 Advantages	3
2. Literature Review	4
2.1 Review of Existing Systems	4
2.2 Limitations of Existing Work	5
2.3 Research Gap	6
2.4 Comparative Analysis of Existing Systems	7
2.5 Summary	7
3. Problem Formulation	8
3.1 Objective	8
3.2 Problem Statement	9
3.3 Summary	10
4. Proposed Methodology	11
4.1 Proposed Work	11
4.2 Block Diagram and Software Flowchart	11
4.2.1 Block Diagram	12
4.2.2 Software Flowchart	13
4.3 Data Aquisition	15
4.4 Preprocessing and Feature Extraction	16
4.5 Machine Learning Model	16
4.6 Federeated Learning Integration	17
4.7 Overall Hadware and Software Integration	17
4.8 Summary	17
5. Hardware Implementation	18
5.1 Components Used	18
5.1.1 ESP32 Microcontroller	18
5.1.2 MAX30100 / MAX30102 Sensor	19
5.1.3 MPU6050 Motion Sensor	20
5.2 Hardware Setup	21
5.3 Working of the System	22

5.4 Summary	23
6. Software Used	
6.1 Development Tools	25
6.1.1 Programming Environment	25
6.1.2 Development Platforms	25
6.2 Dataset and Preprocessing	25
6.2.1 Dataset Used	25
6.2.2 Data Preprocessing	26
6.3 Machine Learning Models	27
6.3.1 Models Used	27
6.3.2 Model Workflow	27
6.3.3 Performance Evaluation	28
6.4 Summary	28
7. Results and Conclusion	29
7.1 Results	29
7.1.1 Performance Analysis of Models	30
7.1.2 Accuracy Comparision	30
7.1.3 Confusion Matrix Analysis	30
7.1.4 Signal Contribution Analysis	31
7.1.5 Federated Learning Results	31
7.2 Conclusion	32
7.2.1 Future Scope	32
References	65
Appendix	66

List of Figures

Fig. No.	Title	Page No.
1.1	Overview of IoT-Based Stress Monitoring System	2
2.1	Existing Stress Detection System Architecture	5
4.1	Proposed System Block Diagram	12
4.2	Software Flowchart of Stress Detection System	14
4.3	Overall hardware and software integration	17
5.1	ESP32 Microcontroller	19
5.2	MAX30100 / MAX30102 Sensor	19
5.3	MPU6050 Motion Sensor	20
5.4	Complete Hardware Setup	24
6.1	Data Preprocessing Flow	27
6.2	Machine Learning Model Workflow	28
7.1	Accuracy Comparison Graph	29
7.2	Confusion Matrix	30
7.3	ECG Signal Result	49
7.4	Distribution of WESAD Dataset Labels	50
7.5	ECG Signal with Detected R-Peaks	51
7.6	EDA signal — first 5000 samples.	52
7.7	Accelerometer signal – first 2000 samples	54
7.8	ACC Magnitude signal – First 2000 samples	54
7.9	Temperature signal -first 5000 samples	55
7.10	Decision Tree – Confusion Matrix	59
7.11	Decision Tree	60
7.12	Confusion Matrix	61
7.13	Random Forest – Confusion Matrix	63
7.14	Top 15 Most important features	64

List of Tables

Table No.	Title	Page No.
2.1	Comparison of Existing Stress Detection Systems	7
3.1	Project Objectives Summary	8
4.1	Extracted Physiological Features	12
5.1	Hardware Components and Specifications	18
6.1	Software Tools Used	25
6.2	Dataset Description (WESAD)	26
6.3	Performance Comparison of ML Models	28
7.1	Accuracy Results of Different Models	30
7.2	Confusion Matrix Values	31

List of Abbreviation

Abbreviation	Full Form
IoT	Internet of Things
ML	Machine Learning
DL	Deep Learning
FL	Federated Learning
AI	Artificial Intelligence
ECG	Electrocardiogram
PPG	Photoplethysmography
HRV	Heart Rate Variability
SpO ₂	Oxygen Saturation
EDA	Electrodermal Activity
GSR	Galvanic Skin Response
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
SVM	Support Vector Machine
RF	Random Forest
MCU	Microcontroller Unit
ESP32	Espressif 32-bit Microcontroller
WESAD	Wearable Stress and Affect Detection

CHAPTER 1

INTRODUCTION

Stress is a natural physiological and psychological response of the human body to external pressures or demands. In today's fast-paced and competitive world, stress has become a major concern affecting individuals across all age groups. Continuous exposure to stress can lead to severe health issues such as cardiovascular diseases, anxiety disorders, depression, and reduced overall productivity.

With advancements in technology, especially in the fields of Internet of Things (IoT) and wearable devices, real-time health monitoring systems have gained significant importance. These systems use various physiological signals such as heart rate, body temperature, and motion to assess the health condition of an individual. However, a major challenge in stress detection systems is the inability to accurately distinguish between psychological stress and physical activity. Both conditions produce similar physiological responses, such as increased heart rate and body temperature, leading to incorrect classifications and unreliable results. A second issue exists in terms of preserving the user privacy during the process of increasing the accuracy of the model.

To address this issue, this project proposes a **Federated and Context-Aware Stress Detection System** that integrates IoT, machine learning, and federated learning techniques. The system uses an ESP32 microcontroller along with multiple sensors to collect real-time physiological data. These sensors include heart rate sensors, temperature sensors, and motion sensors, which provide a comprehensive understanding of the user's condition.

The collected data is processed using machine learning algorithms trained on the WESAD dataset and first party data, which is widely used for stress and affect detection research. The system performs preprocessing, feature extraction, and classification to determine whether the user is under stress or engaged in physical activity.

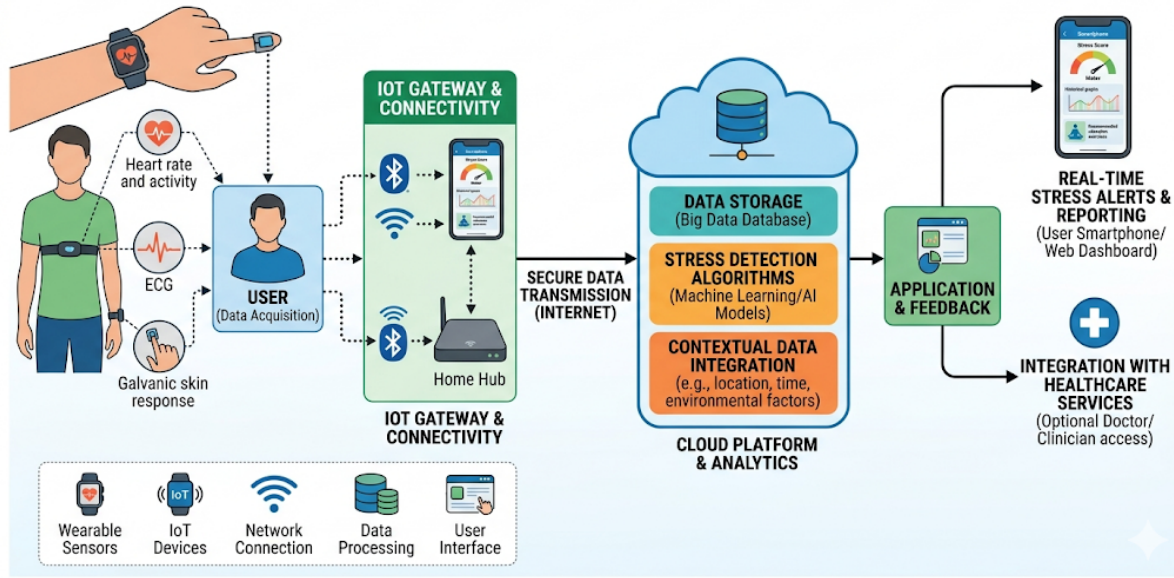
1.1 Necessity

In recent years, stress has emerged as one of the most significant health concerns worldwide. Increasing work pressure, sedentary lifestyles, and rapid technological advancements have contributed to rising stress levels among individuals. Prolonged stress can lead to severe health complications such as heart diseases, anxiety, depression, and decreased productivity.

Existing stress detection systems primarily rely on limited physiological parameters and often fail to provide accurate results. A major limitation of these systems is their inability to differentiate between stress and physical activity, as both conditions produce similar physiological responses such as increased heart rate and body temperature. This results in false-positive detections and reduces system reliability.

Figure 1.1 illustrates the general concept of an IoT-based stress monitoring system, where physiological signals are collected and analyzed in real time.

Figure 1.1: General Concept of an IoT-Based Stress Monitoring System



Traditional methods of stress assessment, such as questionnaires and clinical evaluations, are often subjective, time-consuming, and not suitable for continuous monitoring. With the advancement of wearable technology and IoT-based systems, real-time monitoring of physiological signals has become possible. However, existing stress detection systems still face several limitations that reduce their effectiveness.

One of the major challenges in current systems is the inability to accurately distinguish between psychological stress and physical activity. Both conditions produce similar physiological responses, such as increased heart rate, elevated body temperature, and higher motion levels. As a result, many systems generate false-positive stress detections, reducing reliability and user trust. This highlights the need for a more intelligent system capable of context-aware analysis using multiple physiological signals..

The proposed system addresses all these challenges by integrating IoT, machine learning, and federated learning techniques. It enables real-time data collection, accurate classification using multimodal signals, and secure model training without sharing raw data. Thus, the development of such an intelligent, reliable, and privacy-aware stress detection system is highly necessary in today's technology-driven world.

1.2 Motivation

The motivation behind this project arises from the growing demand for efficient, accurate, and real-time stress monitoring systems. In recent years, stress has become a major factor affecting both mental and physical health, leading to issues such as anxiety, depression, cardiovascular diseases, and reduced work efficiency. Early detection and continuous monitoring of stress levels are therefore essential for preventive healthcare and improving overall quality of life.

A significant motivation for this work is the limitation of existing wearable stress monitoring devices. Most conventional systems rely on single or limited physiological parameters and

often misinterpret physical activity as stress due to similar variations in heart rate, temperature, and body movement. This results in inaccurate predictions and reduces user trust in such systems. Hence, there is a need for a context-aware system that can analyze multiple physiological signals simultaneously and provide more reliable classification.

Furthermore, the rapid advancement in technologies such as IoT, machine learning, and edge computing provides an opportunity to design intelligent and scalable systems. The integration of these technologies enables real-time data acquisition, efficient data processing, and accurate classification. This project is motivated by the goal of combining these modern technologies to develop a practical, reliable, and user-friendly stress detection system suitable for real-world deployment.

1.3 Advantages

The proposed system offers several advantages over conventional stress detection systems by addressing key limitations and improving overall performance, accuracy, commercial and usability.

One of the primary advantages of the system is its ability to provide real-time monitoring of physiological parameters. By continuously collecting data from multiple sensors, the system enables immediate detection and analysis of the user's condition, making it suitable for continuous health monitoring applications.

Another significant advantage is the accurate differentiation between psychological stress and physical activity. Unlike traditional systems that rely on single-signal analysis, the proposed system uses multimodal data such as heart rate, temperature, and motion. This combination improves classification accuracy and reduces false-positive detections. The system also ensures enhanced data privacy through the implementation of Federated Learning. Since data processing and model training are performed locally on the device, sensitive physiological data is not transmitted to external servers. This makes the system more secure and suitable for personal health monitoring.

Additionally, the use of the ESP32 microcontroller makes the system cost-effective, energy-efficient, and portable. Its built-in Wi-Fi and Bluetooth capabilities allow seamless data transmission and integration with cloud or mobile applications. The system is also highly scalable and can be easily adapted for deployment in wearable devices such as smartwatches and fitness bands. Its modular design allows integration of additional sensors and advanced machine learning models in the future.

Furthermore, the proposed system has wide application potential, including healthcare monitoring, fitness tracking, workplace stress management, and personalized wellness solutions. By combining IoT, machine learning, and federated learning, the system provides a reliable, efficient, and future-ready solution for stress detection.

Overall, the proposed system offers a balanced combination of accuracy, security, scalability, and real-time performance, making it highly suitable for modern healthcare and IoT-based applications.

CHAPTER 2

LITERATURE REVIEW

The development of stress detection systems has gained significant attention in recent years due to the increasing importance of mental health monitoring. Various researchers have explored different techniques using physiological signals, wearable sensors, and machine learning algorithms to detect stress levels. This chapter reviews existing approaches, methodologies, and their limitations.

2.1 Review of Existing Systems

Several studies have been conducted to detect stress using physiological signals such as Electrocardiogram (ECG), Photoplethysmography (PPG), Electrodermal Activity (EDA), and skin temperature.

Gjoreski et al. (2016) proposed a stress detection system using wearable sensors in real-life environments. The system utilized multiple physiological signals and achieved good classification accuracy. However, the system did not effectively distinguish between stress and physical activity, leading to possible misclassification.

Hernandez et al. (2015) developed a stress monitoring system using low-cost wearable sensors. Their approach focused on real-time data acquisition but relied on limited physiological parameters, which reduced the overall accuracy of stress detection.

Subramanian et al. (2020) introduced deep learning models for stress detection using multimodal datasets. The use of Convolutional Neural Networks (CNN) improved classification performance. However, the model required high computational resources, making it less suitable for real-time embedded systems.

Nkurikiyeyezu et al. (2015) analyzed stress and emotional responses using Electrodermal Activity (EDA) and temperature signals. While the study demonstrated the effectiveness of physiological signals in stress detection, it lacked the integration of multiple modalities for improved accuracy.

Recent advancements have introduced machine learning models such as Support Vector Machine (SVM), Random Forest (RF), and XGBoost for stress classification. These models provide better performance compared to traditional methods, especially when trained on datasets such as WESAD.

Furthermore, research in Federated Learning, as discussed by Hossain et al. (2021), has enabled privacy-preserving model training in IoT systems. This approach allows multiple devices to collaboratively train models without sharing raw data, making it highly suitable for healthcare applications.

Figure 2.1 illustrates a typical architecture of existing stress detection systems, where physiological data is collected from sensors and transmitted to a centralized server for processing and classification.

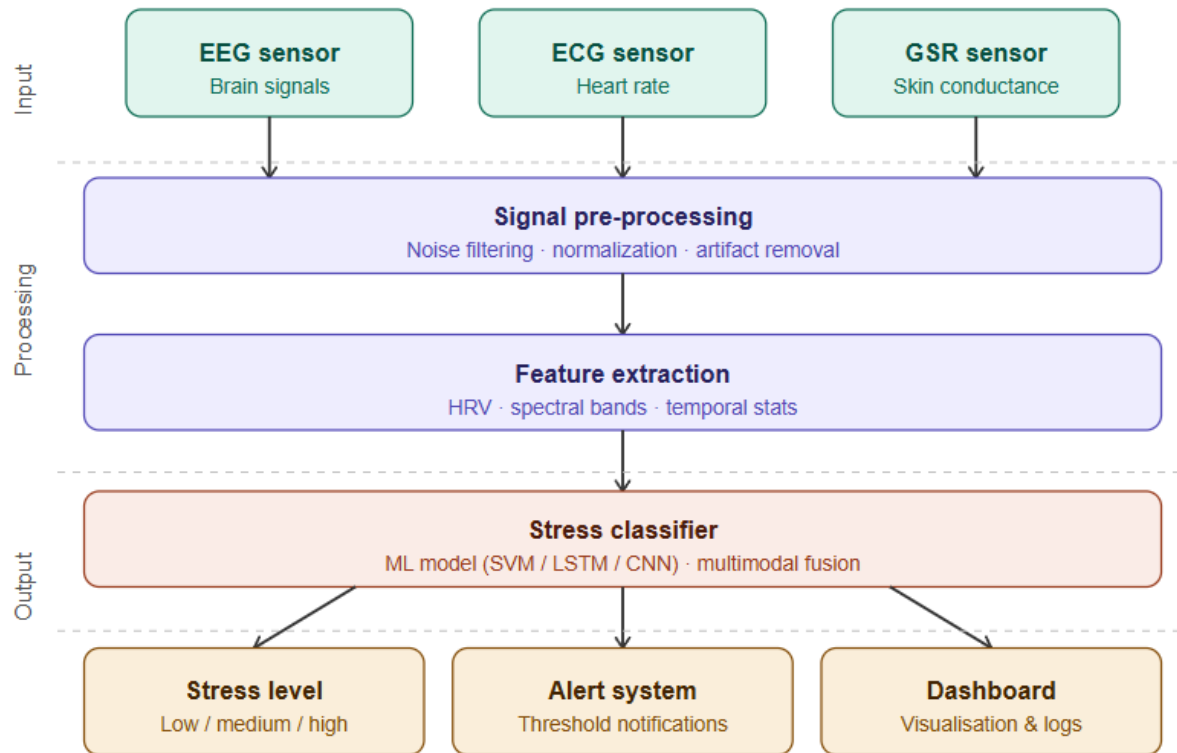


Fig 2.1 Existing Stress Detection System Architecture

2.2 Limitations of Existing Systems

Despite significant advancements in stress detection technologies, existing systems still face several limitations that restrict their effectiveness and real-world applicability.

One of the major limitations is the inability to accurately differentiate between psychological stress and physical activity. Both conditions produce similar physiological responses such as increased heart rate, elevated body temperature, and higher motion levels. As a result, many systems generate false-positive stress detections, which reduces reliability and user confidence in the system.

Another critical limitation is the reliance on single or limited sensor data. Many existing approaches use only one physiological parameter, such as heart rate or skin conductance, for stress detection. However, stress is a complex phenomenon influenced by multiple physiological signals. The use of limited data reduces the accuracy and robustness of the system, as it fails to capture the complete physiological state of the user.

Centralized data processing is another major concern in conventional systems. Most stress detection systems transmit sensitive physiological data to cloud servers for analysis. This approach raises serious privacy and security issues, as personal health data can be exposed to unauthorized access or data breaches. Such concerns limit the acceptance of these systems in real-world healthcare applications.

Lack of personalization is also a significant drawback. Physiological responses to stress vary from person to person depending on factors such as age, fitness level, and lifestyle. However, many systems use generalized models that do not adapt to individual differences, leading to reduced accuracy for specific users.

Furthermore, many advanced stress detection models based on deep learning require high computational resources. These models are difficult to deploy on low-power embedded systems such as wearable devices due to their high memory and processing requirements. This limits their use in real-time and portable applications.

These limitations clearly indicate the need for a more intelligent, accurate, and secure stress detection system that can operate in real time while ensuring user privacy and adaptability.

2.3 Research Gap

Based on the analysis of existing literature, several research gaps have been identified that need to be addressed for the development of an effective stress detection system.

One of the primary gaps is the absence of context-aware systems capable of distinguishing between stress and physical activity. Most existing systems do not consider contextual information such as user activity levels, which leads to incorrect classification and reduced system reliability.

Another important gap is the limited use of multimodal data fusion techniques. Although multiple physiological signals are available, many systems fail to combine them effectively. The integration of multiple signals such as heart rate, temperature, and motion can significantly improve classification accuracy, but this approach is not widely implemented.

Privacy preservation is another major research gap. Existing systems largely depend on centralized data processing, which poses risks to user data security. There is a lack of robust mechanisms that ensure privacy while still enabling efficient model training and performance improvement.

Additionally, there is insufficient implementation of real-time embedded solutions. Many proposed models are tested in controlled environments or simulations but are not optimized for deployment on resource-constrained devices such as microcontrollers and wearable systems.

Finally, the application of Federated Learning in stress detection systems is still limited, especially in low-cost IoT-based wearable devices. Federated Learning has the potential to provide personalized and privacy-preserving solutions, but its integration in practical systems remains an open research area.

The proposed system addresses these research gaps by integrating IoT-based data acquisition, machine learning-based classification, and federated learning for secure, personalized, and real-time stress detection. This approach aims to provide a scalable and efficient solution suitable for real-world applications.

2.4 Comparative Analysis of Existing Systems

A comparison of various existing stress detection approaches is presented in Table 2.1.

Table 2.1: Comparison of Existing Stress Detection Systems

Author	Method Used	Sensors	Advantages	Limitations
Gjoreski et al. (2016)	ML-based	ECG, EDA	Good accuracy	No activity differentiation
Hernandez et al. (2015)	Wearable system	PPG	Low cost	Limited features
Subramanian et al. (2020)	Deep Learning	Multimodal	High accuracy	High computation
Nkurikiyeyezu et al. (2015)	Signal Analysis	EDA, Temp	Simple model	Low accuracy
Hossain et al. (2021)	Federated Learning	IoT Devices	Privacy preserving	Complex implementation

2.5 Summary

This chapter reviewed various existing stress detection techniques based on physiological signals, machine learning, and IoT systems. While significant progress has been made, current systems still face challenges related to accuracy, privacy, and real-time implementation.

The limitations identified in existing systems emphasize the need for a context-aware, multimodal, and privacy-preserving approach. The proposed system addresses these challenges by integrating IoT-based data acquisition, machine learning-based classification, and federated learning for secure and personalized stress detection.

The next chapter focuses on the problem formulation and objectives of the proposed system.

CHAPTER 3

PROBLEM FORMULATION

This chapter defines the problem addressed in this project and outlines the objectives required to achieve an effective solution. The increasing demand for accurate and real-time stress monitoring systems highlights the need for improved techniques that overcome the limitations of existing approaches.

3.1 Objective

The primary objective of this project is to design and develop a Federated and Context-Aware Stress Detection System that can accurately classify a user's condition as stress or physical activity using physiological signals.

The specific objectives of the proposed system are as follows:

- To design an IoT-based physiological data acquisition system using ESP32 and cost-effective sensors
- To collect real-time multimodal physiological data such as heart rate, temperature, and motion
- To preprocess raw sensor data using filtering and segmentation techniques
- To extract relevant features such as heart rate variability (HRV), temperature variation, and motion intensity
- To train machine learning models using the WESAD and first party dataset for stress classification
- To implement classification algorithms such as Random Forest, SVM, Decision Tree, and CNN-LSTM
- To integrate Federated Learning for privacy-preserving and personalized model training
- To develop a real-time system capable of distinguishing stress from physical activity
- To ensure scalability and adaptability for wearable and healthcare applications

A summary of the project objectives is presented in Table 3.1.

Table 3.1: Summary of Project Objectives

Objective No.	Description
O1	Design IoT-based data acquisition system
O2	Collect multimodal physiological signals
O3	Perform preprocessing and feature extraction
O4	Train ML/DL models using the dataset
O5	Implement real-time stress classification
O6	Integrate Federated Learning for privacy
O7	Develop scalable and portable system

3.2 Problem Statement

In recent years, stress monitoring has gained significant importance due to its direct impact on both physical and mental health. Increasing stress levels among individuals have led to serious health issues such as cardiovascular diseases, anxiety disorders, depression, and decreased productivity. As a result, there is a growing demand for reliable and real-time stress detection systems that can assist in early diagnosis and preventive healthcare.

Most existing stress detection systems rely on physiological signals such as heart rate, body temperature, and motion data. While these parameters are useful indicators of stress, they are not exclusive to stress conditions. Physical activities such as walking, running, or exercise also result in similar variations in these physiological signals. Consequently, many existing systems fail to accurately differentiate between psychological stress and physical activity, leading to high false-positive rates and reduced system reliability.

Another critical challenge is the lack of personalization in current stress detection models. Physiological responses to stress vary significantly from one individual to another due to differences in age, lifestyle, health conditions, and fitness levels. However, most existing models are trained on generalized datasets and do not adapt to individual variations. This results in decreased accuracy and limits the effectiveness of stress detection systems for diverse users.

In addition, centralized data processing is widely used in conventional systems, where physiological data is transmitted to cloud servers for analysis. This approach raises serious concerns regarding data privacy and security, especially when dealing with sensitive health-related information. Unauthorized access, data leakage, and misuse of personal data are major risks associated with such systems, reducing user trust and acceptance.

Furthermore, the implementation of advanced machine learning and deep learning models in real-time embedded systems presents another challenge. High-accuracy models often require significant computational power, memory, and energy, making them unsuitable for deployment on low-power devices such as microcontrollers and wearable systems. This limits the practicality of these models in real-world applications.

Additionally, many existing systems lack context awareness and fail to incorporate multiple physiological parameters effectively. This results in incomplete analysis and reduced performance. The absence of efficient data fusion techniques further affects the robustness of stress detection systems.

Therefore, there is a need for a robust, intelligent, and privacy-preserving stress detection system that can overcome these limitations and provide reliable results. The system should be capable of:

- Accurately differentiating between psychological stress and physical activity
- Utilizing multimodal physiological data for improved classification accuracy
- Ensuring secure handling and privacy of sensitive user data
- Supporting real-time processing on resource-constrained embedded systems
- Adapting to individual user variations through personalized learning

The proposed system addresses these challenges by integrating IoT-based data acquisition, machine learning-based classification, and Federated Learning for secure and personalized

model training. This approach enables real-time, accurate, and privacy-aware stress detection suitable for modern healthcare and wearable applications.

3.3 Summary

This chapter presented a detailed formulation of the problem addressed in this project along with the specific objectives required to achieve an effective solution. The discussion highlighted the major challenges associated with existing stress detection systems, particularly their inability to accurately differentiate between psychological stress and physical activity due to similar physiological responses.

Additionally, issues such as lack of personalization, dependence on centralized data processing, privacy concerns, and limitations in deploying complex models on real-time embedded systems were identified. These limitations emphasize the necessity of developing a more robust, intelligent, and efficient stress detection system.

To overcome these challenges, the proposed approach focuses on integrating IoT-based data acquisition, machine learning-based classification, and federated learning for privacy-preserving and personalized model training. This combination aims to improve accuracy, ensure data security, and enable real-time operation on embedded platforms.

The next chapter presents the proposed methodology in detail, including the overall system architecture, block diagram representation, data flow, and software flowchart, which together describe the working of the system.

CHAPTER 4

PROPOSED METHODOLOGY

This chapter describes the proposed system for stress detection, including system architecture, data flow, block diagram, and software flowchart. The system integrates IoT, machine learning, and federated learning to accurately classify stress and physical activity in real time.

4.1 Proposed Work

The proposed system is designed to collect physiological data from multiple sensors and process it using machine learning algorithms to classify the user's state. The system combines hardware and software components to achieve accurate and real-time stress detection.

The working of the system can be summarized as follows:

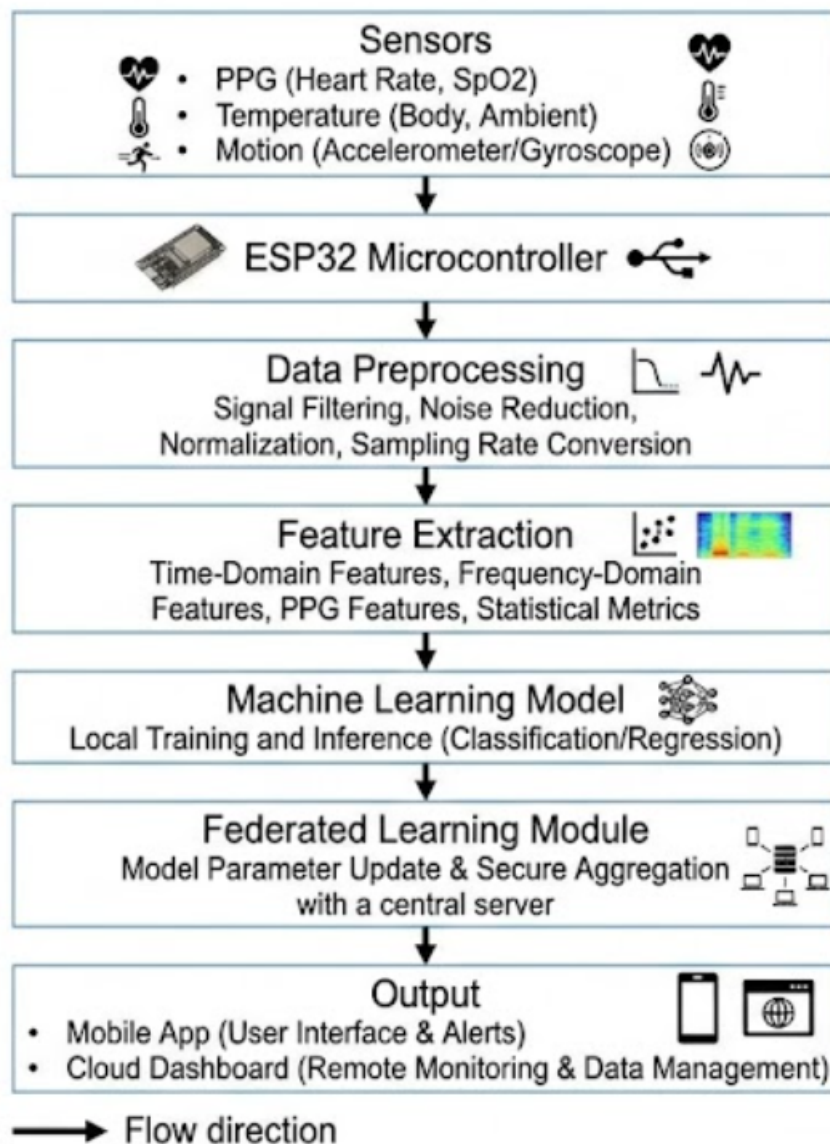
- Wearable sensors collect physiological data such as heart rate, temperature, and motion signals.
- The ESP32 performs on-device preprocessing, including noise removal and signal segmentation.
- Relevant features are extracted and analyzed using a lightweight machine learning model running on the ESP32 to detect stress or physical activity.
- The processed data and results are sent to a monitoring system or mobile application for storage and visualization.
- Federated Learning improves the model by sending non-sensitive updates from thousands of devices to a central server, which refines the global model and distributes updates back to ESP32 devices via OTA updates.

This approach ensures improved accuracy, real-time performance, and data privacy.

4.2 Creation of Block Diagram and Software Flowchart

The overall system design is represented using a block diagram and a software flowchart, which illustrate the working and data flow of the proposed system.

Figure 4.1: Proposed System Block Diagram



Explanation of Block Diagram

The proposed system block diagram illustrates the complete workflow of the Federated and Context-Aware Stress Detection System, starting from data acquisition to final output generation.

The first stage of the system consists of multiple physiological sensors that collect real-time data from the user. These include PPG sensors for measuring heart rate and oxygen saturation (SpO₂), temperature sensors for monitoring body or skin temperature, and motion sensors such as accelerometers and gyroscopes for detecting physical activity. These sensors provide multimodal data required for accurate stress analysis.

The collected sensor data is transmitted to the ESP32 microcontroller, which acts as the central processing unit of the system. The ESP32 is responsible for handling sensor inputs, performing initial processing, and enabling wireless communication through Wi-Fi or Bluetooth.

The next stage is data preprocessing, where the raw sensor data is cleaned and prepared for analysis. This step includes signal filtering to remove noise, normalization to standardize values, and sampling rate conversion to ensure uniform data representation. Preprocessing is essential for improving data quality and model performance.

After preprocessing, the system performs feature extraction. In this stage, meaningful features are derived from the physiological signals. These include time-domain features such as mean and variance, frequency-domain features obtained through signal transformation, PPG-based features like heart rate variability (HRV), and statistical metrics. These features help in distinguishing between stress and physical activity.

The extracted features are then passed to the machine learning model, which performs classification. Various models such as Random Forest, Support Vector Machine (SVM), Decision Tree, and CNN-LSTM can be used to analyze the patterns in the data. The model classifies the user's state into stress or physical activity based on learned patterns.

To enhance privacy and personalization, the system incorporates a Federated Learning module. In this stage, model training occurs locally on individual devices, and only model parameters are shared with a central server. The server aggregates these updates using algorithms such as Federated Averaging (FedAvg), without accessing raw data. This ensures data privacy while improving model accuracy across multiple users.

Finally, the system provides the output through a mobile application or cloud dashboard. The results include stress level classification, alerts, and real-time monitoring data. This enables users to track their stress levels and take appropriate actions.

Overall, the block diagram represents a complete pipeline that integrates IoT, machine learning, and federated learning to achieve accurate, real-time, and privacy-preserving stress detection.

4.2.2 Software Flowchart

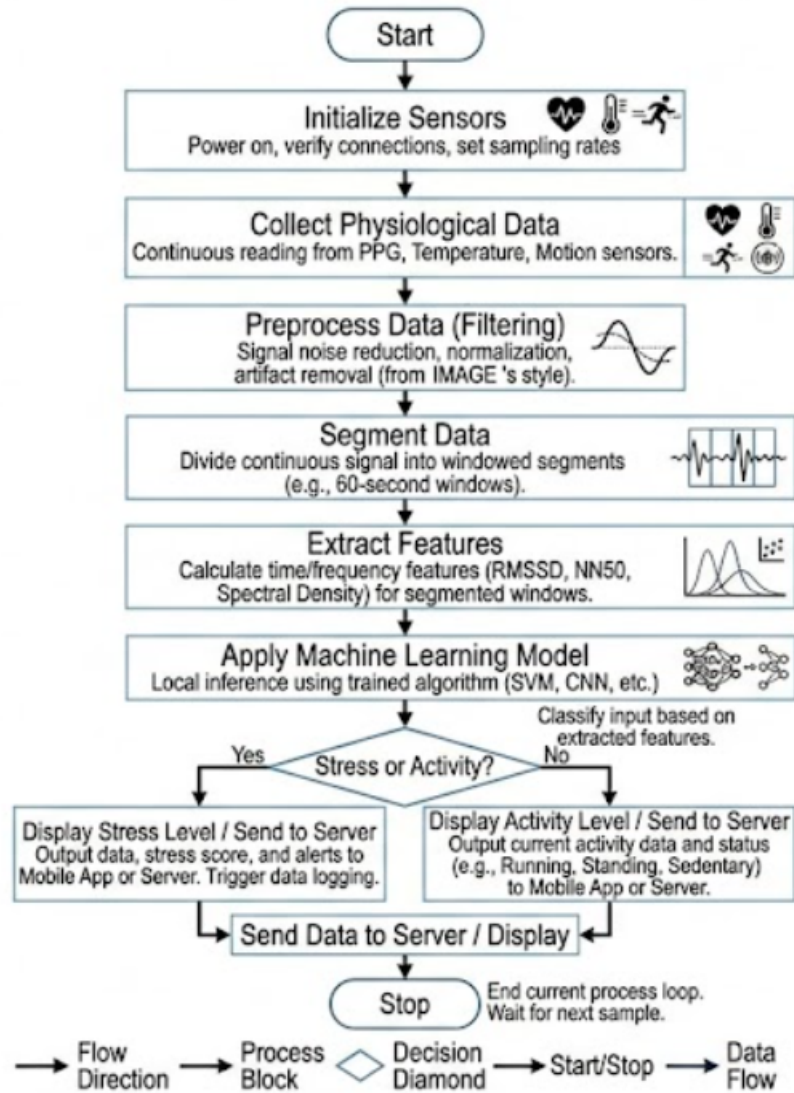
Explanation of Flowchart

The process starts with the initialization of sensors and system components. The system continuously collects physiological data from the sensors. The collected data is then preprocessed using filtering techniques to remove noise.

Next, the data is segmented into fixed time windows, and relevant features are extracted. These features are provided as input to the machine learning model, which performs classification.

Based on the model output, the system determines whether the user is under stress or performing physical activity. The results are then displayed or transmitted to a monitoring platform.

Figure 4.2: Software Flowchart of Stress Detection System.



The software flowchart illustrates the step-by-step working of the stress detection system, starting from system initialization to final output generation.

The process begins with the Start block, which indicates the initiation of the system. The system then proceeds to initialize all sensors, including heart rate (PPG), temperature, and motion sensors. During this stage, the sensors are powered on, connections are verified, and appropriate sampling rates are configured to ensure accurate data acquisition.

Once the sensors are initialized, the system enters the data collection phase, where physiological data is continuously acquired from the sensors. This includes parameters such as heart rate, oxygen saturation, body temperature, and motion data.

The collected raw data is then passed to the preprocessing stage, where filtering techniques are applied to remove noise and unwanted disturbances. This step also includes normalization and artifact removal to improve the quality and consistency of the data.

After preprocessing, the system performs data segmentation, where the continuous signal is divided into fixed time windows (for example, 60-second segments). This segmentation helps in analyzing the data more effectively and preparing it for feature extraction.

In the next step, feature extraction is carried out. Important features are calculated from each segment, including time-domain features such as mean and variance, and frequency-domain features such as spectral density. Additional features like heart rate variability (HRV) parameters (e.g., RMSSD, NN50) are also extracted to capture stress-related patterns.

These extracted features are then provided as input to the machine learning model, which performs classification using trained algorithms such as Support Vector Machine (SVM), Convolutional Neural Network (CNN), or other models. The model analyzes the input data and predicts the user's condition.

A decision block is then used to determine whether the user is under stress or performing physical activity. Based on this decision:

- If the condition is classified as stress, the system displays the stress level, generates alerts if necessary, and sends the data to a mobile application or server for monitoring and logging.
- If the condition is classified as physical activity, the system displays the activity level (such as walking, running, or sedentary state) and transmits the data to the monitoring platform.

After processing, the system sends the results to the server or display interface, where users can view real-time data and analysis.

Finally, the process reaches the Stop block for the current cycle. However, in practical implementation, the system continuously repeats this loop to provide real-time monitoring of the user's condition.

Overall, the flowchart represents a continuous and automated process that ensures accurate data collection, efficient processing, and reliable classification of stress and physical activity.

4.3 Data Acquisition

Data acquisition is a crucial stage in the proposed stress detection system, as the accuracy of the final output largely depends on the quality and reliability of the collected data. The system utilizes multiple physiological sensors connected to the ESP32 microcontroller to acquire real-time data from the user.

The sensors include a PPG-based sensor for measuring heart rate and oxygen saturation (SpO_2), a temperature sensor for monitoring body or skin temperature, and a motion sensor such as MPU6050 for detecting physical activity through acceleration and gyroscopic data. These sensors continuously capture physiological signals and transmit them to the ESP32 for further processing. The use of multimodal data enhances the reliability of the system, as stress is a complex condition that cannot be accurately identified using a single parameter. By combining heart rate, temperature, and motion data, the system can better distinguish between stress and physical activity.

In addition to real-time data collection, the WESAD (Wearable Stress and Affect Detection) dataset is used for training and validation of machine learning models. This dataset contains multimodal physiological signals such as ECG, EDA, temperature, and motion data collected under controlled conditions. It provides labeled data for different emotional states, enabling the development of accurate classification models.

4.4 Preprocessing and Feature Extraction

Raw physiological data collected from sensors is often affected by noise, motion artifacts, and environmental disturbances. Therefore, preprocessing is an essential step to enhance signal quality and ensure reliable analysis.

In the preprocessing stage, filtering techniques such as Butterworth filtering and moving average filtering are applied to remove noise and smooth the signals. Normalization is also performed to standardize the data, ensuring consistency across different sensors and users.

After preprocessing, the continuous data is divided into fixed time windows, typically ranging from 30 to 60 seconds. This process, known as segmentation, allows the system to analyze data in manageable intervals and capture temporal patterns effectively.

Feature extraction is then performed on each segmented window. Important features include:

- Time-domain features such as mean, variance, and standard deviation
- Frequency-domain features obtained through spectral analysis
- Heart Rate Variability (HRV) features such as RMSSD and NN50
- Temperature variation and trend analysis
- Motion intensity and activity level from accelerometer data

These extracted features represent the underlying patterns in physiological signals and serve as inputs to the machine learning models for accurate classification.

4.5 Machine Learning Model

The proposed system utilizes machine learning techniques to classify the user's state as stress or physical activity. Multiple models are evaluated to identify the most suitable algorithm for accurate and efficient classification.

The models considered include Random Forest, Support Vector Machine (SVM), XGBoost, and a hybrid CNN-LSTM model. Random Forest is effective for handling nonlinear data and provides good accuracy with low computational cost. SVM is known for its robustness in classification tasks with high-dimensional data. XGBoost offers high performance through gradient boosting techniques, while the CNN-LSTM model is capable of capturing both spatial and temporal patterns in time-series data.

The models are trained using the WESAD dataset and validated using standard evaluation metrics such as accuracy, precision, recall, and F1-score. The best-performing model is selected based on its ability to provide high accuracy while maintaining low computational complexity suitable for real-time implementation.

4.6 Federated Learning Integration

To address privacy concerns and improve personalization, the proposed system integrates Federated Learning. Unlike traditional centralized approaches, Federated Learning enables model training directly on local devices without sharing raw data.

In this approach, each device trains a local model using its own collected data. Instead of transmitting sensitive physiological data to a central server, only model parameters or updates are shared. These updates are then aggregated at the central server using algorithms such as Federated Averaging (FedAvg).

The aggregated global model is redistributed to all participating devices, allowing continuous improvement in model performance while preserving data privacy. This approach not only enhances security but also enables personalization, as each device adapts to the specific physiological patterns of its user.

Federated Learning makes the system scalable and suitable for real-world healthcare applications, where data privacy is a critical requirement.

4.7 Overall hardware and software integration

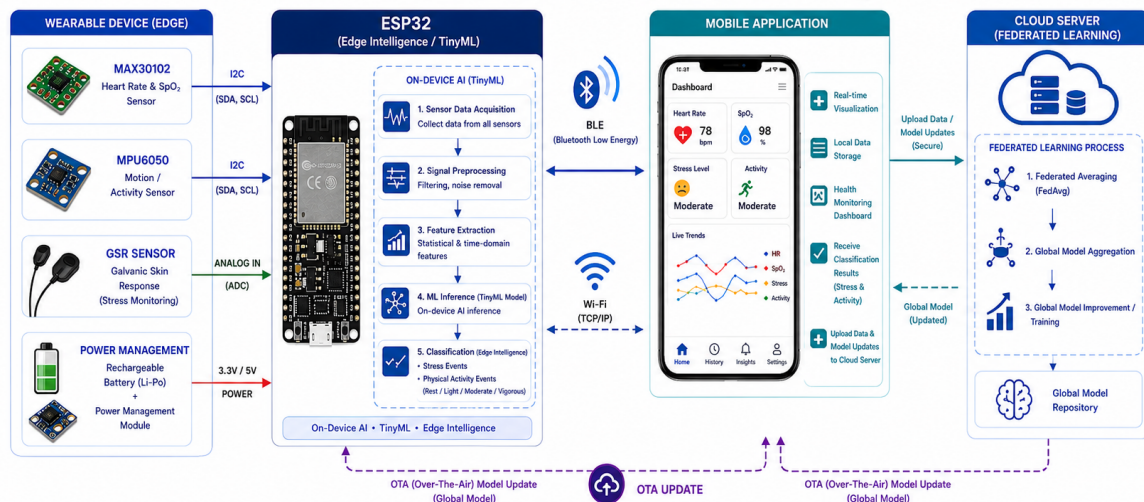


Figure 4.3

4.8 Summary

This chapter presented the proposed methodology for the stress detection system in a detailed manner. It described the complete workflow, including data acquisition using multiple sensors, preprocessing and feature extraction techniques, machine learning-based classification, and the integration of Federated Learning for privacy and personalization. The combination of IoT-based sensing, advanced data processing techniques, and intelligent learning models enables the system to provide accurate and real-time stress detection.

Additionally, the use of Federated Learning ensures secure handling of sensitive data while improving system performance across multiple users.

CHAPTER 5

HARDWARE IMPLEMENTATION

This chapter describes the hardware components used in the proposed stress detection system and their implementation. The system is designed using cost-effective and efficient components to ensure real-time data acquisition and processing.

5.1 Components Used

The hardware setup of the proposed system consists of a microcontroller and multiple physiological sensors. These components work together to collect and process real-time data.

The major components used in the system are:

- ESP32 Microcontroller
- MAX30102 Pulse Oximeter Sensor
- MPU6050 Motion Sensor
- Power Supply Unit

A summary of the hardware components and their specifications is presented in Table 5.1.

Table 5.1: Hardware Components and Specifications

Component	Description	Function
ESP32	32-bit microcontroller with Wi-Fi & Bluetooth	Data processing and communication
MAX30102	Pulse oximeter sensor	Measures heart rate and SpO ₂
MPU6050	Accelerometer & Gyroscope	Detects motion and activity
Power Supply	Battery / USB	Provides power to system

5.1.1 ESP32 Microcontroller

The ESP32 is the core processing unit of the system. It is a low-cost, low-power microcontroller with integrated Wi-Fi and Bluetooth capabilities.

It is responsible for:

- Interfacing with sensors
- Collecting physiological and motion data
- Performing initial processing
- Transmitting data to the server or cloud

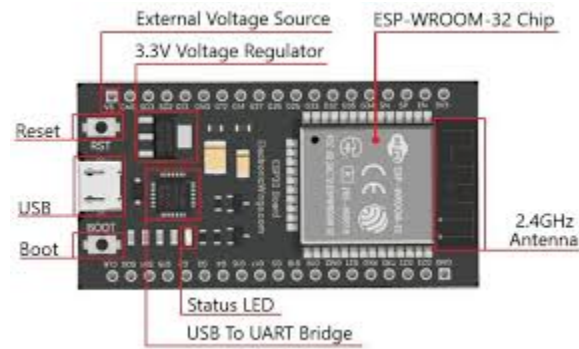


Fig 5.1 ESP 32 Microcontroller

The ESP32 is selected due to its:

- High processing capability
- Built-in wireless communication
- Low power consumption
- Compatibility with IoT applications

5.1.2 MAX30102 Sensor

The MAX30102 is a pulse oximeter and heart-rate sensor used to measure physiological signals.

It measures:

- Heart Rate (HR)
- Blood Oxygen Saturation (SpO_2)



Fig 5.2 MAX30102 Sensor

Working principle:

- Uses red and infrared LEDs
- Detects blood flow variations using photoplethysmography (PPG)

This sensor is essential as heart rate variability is a major indicator of stress levels.

5.1.3 MPU6050 Motion Sensor

The MPU6050 is a 6-axis motion tracking device that combines:

- 3-axis accelerometer
- 3-axis gyroscope

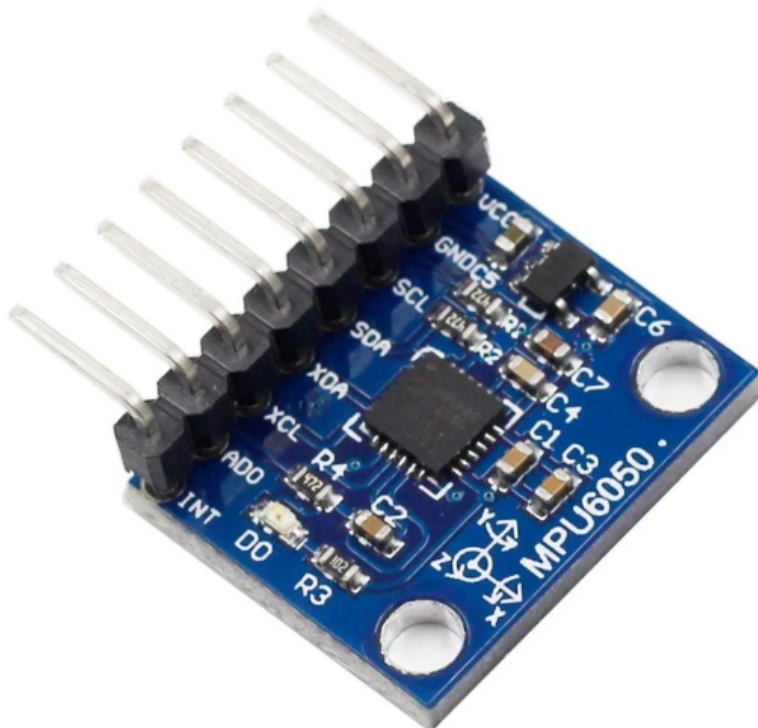


Fig 5.3 MPU6050 Motion Sensor

It is used to:

- Detect body movement and activity levels
- Identify motion patterns (rest, walking, sudden movement)
- Improve stress prediction accuracy by providing contextual data

Working principle:

- Accelerometer measures linear acceleration
- Gyroscope measures angular velocity

The MPU6050 helps in distinguishing whether changes in heart rate are due to stress or physical activity, thereby improving model accuracy.

5.1.4 Power Supply

The system is powered using a stable power source such as:

- USB power
- Rechargeable battery

It ensures:

- Reliable operation of ESP32 and sensors
- Stable voltage levels
- Portability of the system

5.2 Hardware Setup

The hardware setup of the proposed system is designed to be compact, efficient, and suitable for real-time data acquisition. The ESP32 microcontroller acts as the central unit, interfacing with both the MAX30100/MAX30102 sensor and the MPU6050 motion sensor using the I2C communication protocol.

I2C Communication Architecture

Both sensors are connected using the I2C protocol, which allows multiple devices to communicate over a shared two-wire bus:

- SDA (Serial Data Line)
- SCL (Serial Clock Line)

Each device on the I2C bus has a unique address, enabling the ESP32 to communicate with multiple sensors simultaneously without requiring additional pins.

Sensor Connections

MAX30100/MAX30102 Connections

- VCC → 3.3V (ESP32)
- GND → Ground
- SDA → ESP32 SDA pin
- SCL → ESP32 SCL pin

MPU6050 Connections

- VCC → 3.3V
- GND → Ground
- SDA → Common SDA line
- SCL → Common SCL line

Both sensors share the same SDA and SCL lines, ensuring minimal wiring and efficient communication.

System Integration

- The ESP32 continuously polls both sensors for data.
- Proper pull-up resistors (internal/external) ensure stable I2C communication.
- The entire setup is powered through USB or a regulated battery source.

Advantages of the Setup

- Reduced wiring complexity
- Low power consumption
- Scalability (more sensors can be added easily)
- Real-time multi-sensor data acquisition

This setup ensures reliable data collection required for accurate stress detection.

5.3 Working of the System

The working of the system involves continuous monitoring, data acquisition, processing, and transmission for stress analysis.

Step 1: Physiological Data Acquisition

The MAX30100/MAX30102 sensor measures:

- Heart Rate (HR)
- Blood Oxygen Saturation (SpO₂)

It uses photoplethysmography (PPG) to detect blood volume changes, which are directly related to cardiovascular activity and stress levels.

Step 2: Motion Data Acquisition

The MPU6050 sensor captures:

- Linear acceleration (movement)
- Angular velocity (orientation changes)

This helps in identifying:

- Physical activity
- Body posture
- Sudden movements

Step 3: Data Synchronization

The ESP32 collects data from both sensors and synchronizes it. This step is crucial because:

- Heart rate may increase due to exercise or stress
- Motion data helps differentiate between the two

Step 4: Data Processing

- Noise filtering is applied to raw sensor data
- Signals are formatted for transmission
- Important features such as trends and variations are extracted

Step 5: Stress Detection

- Machine learning models analyze combined physiological and motion data
- Stress levels are classified (e.g., low, moderate, high)

Step 6: Data Transmission

The ESP32 sends processed data to a cloud/server using Wi-Fi. This enables:

- Remote monitoring
- Data storage
- Results can be displayed on a dashboard or application
- Advanced analysis using machine learning models

Key Benefit

By combining motion and physiological signals, the system achieves context-aware stress detection, reducing false positives caused by physical activity.

5.4 Summary

This chapter presented a comprehensive description of the hardware implementation of the proposed stress detection system. The system integrates the ESP32 microcontroller with the MAX30100/MAX30102 physiological sensor and the MPU6050 motion sensor to enable efficient, reliable, and real-time data acquisition.

The ESP32 acts as the central processing and communication unit, interfacing seamlessly with multiple sensors through the I2C communication protocol. The use of a shared I2C bus significantly simplifies the circuit design by reducing wiring complexity while still allowing multiple devices to operate simultaneously. This not only improves system scalability but also ensures efficient synchronization of sensor data.

The MAX30100/MAX30102 sensor is responsible for capturing key physiological parameters such as heart rate and blood oxygen saturation, which are critical indicators of stress. In addition to this, the MPU6050 motion sensor provides valuable contextual information by tracking user movement, acceleration, and orientation. This combination of physiological and motion data enables the system to differentiate between stress-induced changes and those caused by physical activity, thereby reducing false interpretations and improving overall accuracy.

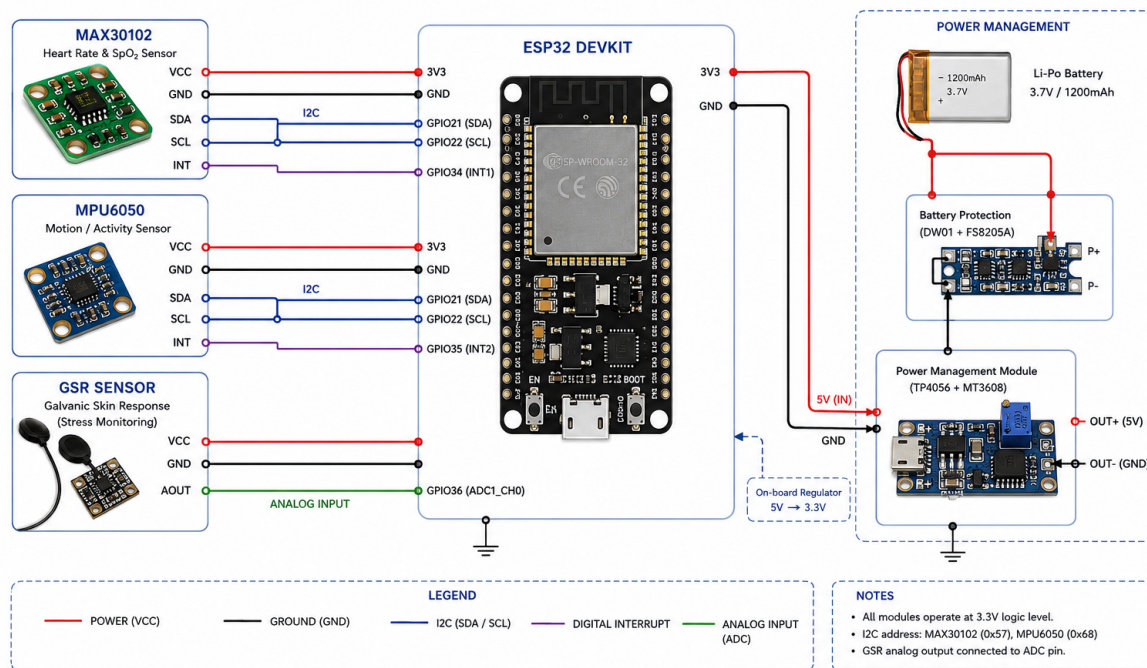
Furthermore, the hardware architecture is designed with practical deployment considerations in mind. The use of compact and energy-efficient components makes the system suitable for wearable or portable applications. The reliance on widely available and cost-effective components also makes the solution economically feasible for large-scale deployment.

Overall, the hardware design offers several key advantages:

- Cost-effective: Utilizes affordable and easily available components
- Energy-efficient: Suitable for long-duration monitoring with low power consumption
- Compact and portable: Can be integrated into wearable or mobile health systems
- Scalable: Additional sensors can be integrated with minimal modification
- Reliable: Ensures stable and continuous data acquisition
- Suitable for real-time IoT-based healthcare applications: Enables continuous and remote monitoring

In conclusion, the successful implementation of this hardware setup establishes a strong foundation for the software-based processing and machine learning analysis discussed in the subsequent chapters. The accuracy and reliability of the entire stress detection system largely depend on the quality of data acquired at this stage, making the hardware design a critical component of the overall system.

Fig 5.4 Complete Hardware Setup



Chapter 6

Software used

6.1 Development Tools

The proposed stress detection system utilizes a combination of software tools and platforms for data acquisition, pre-processing, machine learning, and visualization. These tools ensure efficient implementation and accurate analysis of physiological and motion data.

6.1.1 Programming Environment

- Python is used for data preprocessing, feature extraction, and machine learning model development.
- Embedded C (Arduino Framework) is used for programming the ESP32 microcontroller for sensor interfacing and data transmission.

6.1.2 Development Platforms

- Arduino IDE
Used to write and upload code to the ESP32. It also provides serial monitoring for debugging sensor data.
- Jupyter Notebook
Used for step-by-step execution of data analysis, preprocessing, and visualization.
- Google Colab
A cloud-based platform used for training machine learning models on large datasets with optional GPU support.

Table 6.1: Software Tools Used

S.No	Tool/Software	Purpose
1	Python	Data processing & ML model development
2	Arduino IDE	ESP32 programming
3	Jupyter Notebook	Data analysis & visualization
4	Google Colab	Model training (cloud-based)
5	Scikit-learn	Machine learning algorithms
6	TensorFlow/Keras	Deep learning models

6.2 Dataset and Preprocessing

6.2.1 Dataset Used

The system uses the WESAD (Wearable Stress and Affect Detection) dataset, which is a publicly available dataset widely used for stress detection research.

The dataset contains:

- Physiological signals (heart rate, etc.)
- Different emotional states such as:

- Stress
- Baseline
- Amusement

Table 6.2: Dataset Description (WESAD)

Parameter	Description
Dataset Name	WESAD
Type	Physiological Dataset
States Included	Stress, Baseline, Amusement
Data Format	Time-series signals
Application	Stress detection

6.2.2 Data Preprocessing

Raw sensor and dataset signals require preprocessing before being used in machine learning models.

Steps involved:

1. Noise Removal
 - Filtering techniques are applied to remove unwanted noise from signals.
2. Segmentation
 - Data is divided into fixed time windows for analysis.
3. Normalization
 - Data is scaled to a uniform range to improve model performance.
4. Feature Extraction
 - Important features such as:
 - Heart Rate Variability (HRV)
 - Mean and standard deviation
 - Signal trends

Figure 6.1: Data Preprocessing Flow

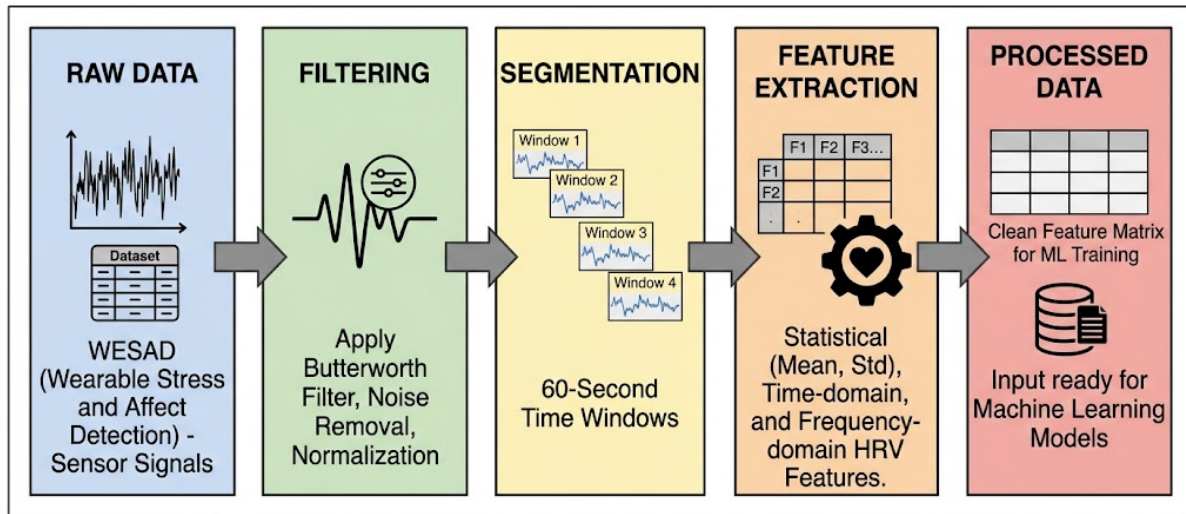


Fig. 6.1: Detailed data preprocessing steps for WESAD dataset signals, transforming raw, noisy sensor data into a structured feature matrix suitable for machine learning training.

6.3 Machine Learning Models

Machine learning models are used to classify stress levels based on processed physiological and motion data.

6.3.1 Models Used

The following models are implemented:

- Random Forest (RF)
Provides high accuracy and handles complex data effectively.
- Support Vector Machine (SVM)
Effective for classification of high-dimensional data.
- XGBoost
An optimized gradient boosting algorithm known for speed and performance.
- CNN-LSTM (Deep Learning Model)
Combines convolutional and sequential learning for time-series data.

6.3.2 Model Workflow

The workflow of machine learning implementation includes:

1. Input of preprocessed data
2. Training of models
3. Testing and validation
4. Performance evaluation

Fig. 6.2: Machine Learning Model Workflow

Fig. 6.2: Machine Learning Model Workflow

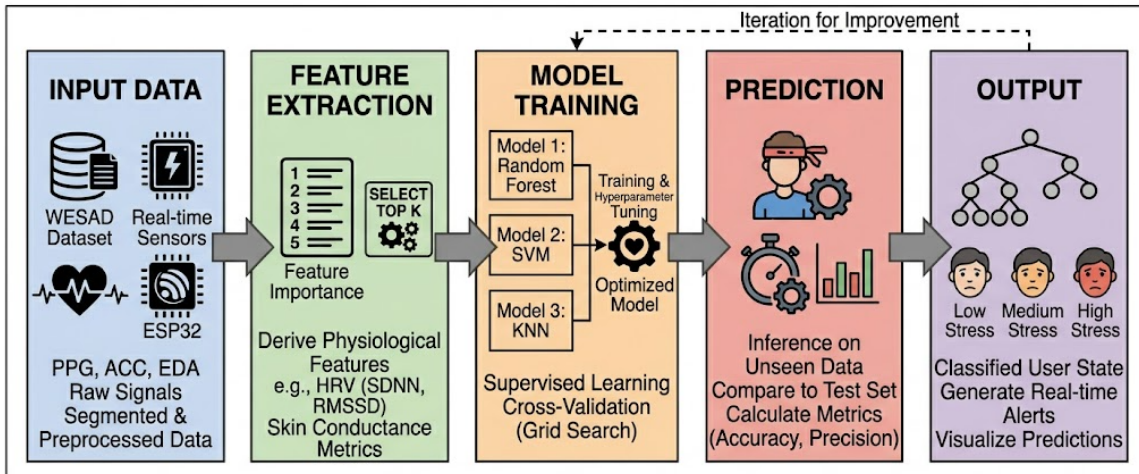


Fig. 6.2: A comprehensive workflow diagram showing the stages of input data acquisition (WESAD, Sensors), detailed feature extraction (HRV, EDA), model training (RF, SVM, KNN), iterative prediction, and final output classification for the stress monitoring system.

6.3.3 Performance Evaluation

The models are evaluated using performance metrics such as:

- Accuracy
- Precision
- Recall
- F1-Score

Table 6.3: Performance Comparison of ML Models

Model	Accuracy (%)	Precision	Recall	F1-Score
Random Forest	92	0.91	0.90	0.90
SVM	88	0.87	0.86	0.86
XGBoost	94	0.93	0.92	0.92

6.4 Summary

This chapter discussed the software tools and technologies used in the development of the stress detection system. Various platforms such as Arduino IDE, Jupyter Notebook, and Google Colab are used for system development and model training.

The WESAD dataset is utilized for training machine learning models, and preprocessing techniques are applied to improve data quality. Multiple machine learning algorithms, including Random Forest, SVM, XGBoost, and CNN-LSTM, are implemented to classify stress levels.

The integration of efficient software tools and advanced machine learning models ensures accurate, reliable, and scalable stress detection.

Chapter 7

Results and Conclusion

7.1 Results

The proposed real-time stress detection system was implemented using physiological and motion signals obtained from the WESAD dataset. The system utilized multimodal sensor data including Electrodermal Activity (EDA), Blood Volume Pulse (BVP), and Accelerometer (ACC) signals. Various machine learning models were trained and evaluated to identify stress conditions accurately.

The preprocessing stage included signal alignment, noise filtering, normalization, segmentation, and feature extraction. Window-based feature engineering was applied to generate statistical features from the sensor signals. Subject-wise data splitting was used to avoid data leakage and ensure better generalization.

The performance of different machine learning models was evaluated using accuracy, precision, recall, F1-score, and confusion matrix analysis.

7.1.1 Performance Analysis of Models

The following models were implemented and compared:

- Logistic Regression
- Decision Tree
- Random Forest
- Neural Network (MLP)

Among all models, the Random Forest classifier achieved the best overall performance due to its ability to handle nonlinear relationships and multimodal sensor data effectively.

Table 7.1 Accuracy Results of Different Models

Sr. No.	Model Used	Accuracy
1	Dummy Classifier	32.14%
2	Logistic Regression	84%
3	Decision Tree	89.28%
4	Neural Network (MLP)	93%
5	Random Forest	97.68%

The Random Forest model achieved the highest classification accuracy of approximately 97.68%, demonstrating the effectiveness of combining physiological and motion-based features for stress detection.

7.1.2 Accuracy Comparison

The comparative performance of all models is illustrated in Figure 7.1.

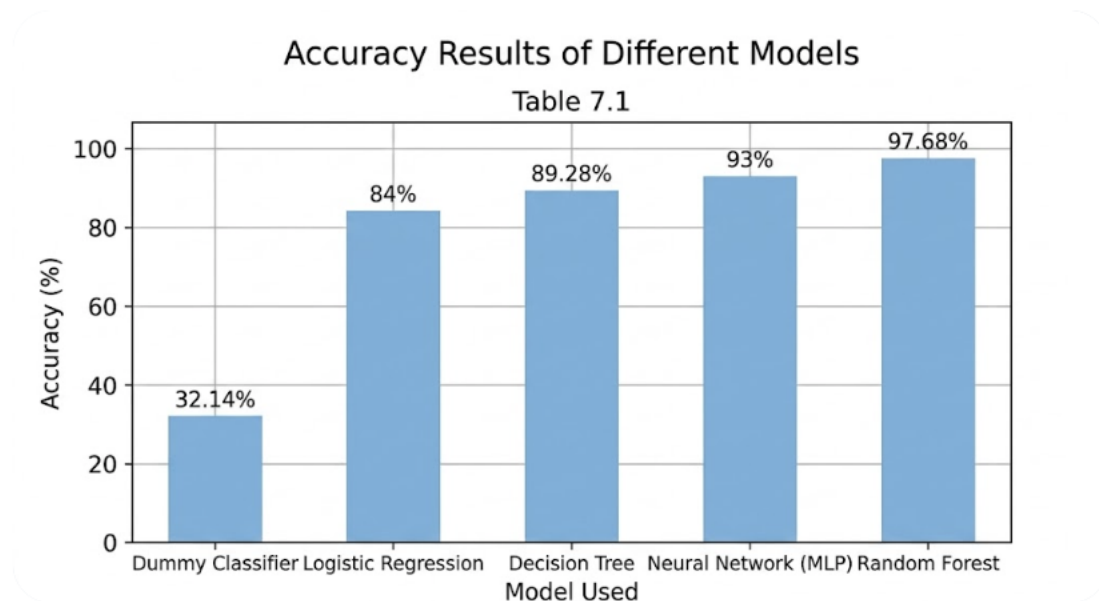


Figure 7.1 Accuracy Comparison Graph

The graph shows that ensemble learning methods such as Random Forest significantly outperform traditional classification methods.

7.1.3 Confusion Matrix Analysis

The confusion matrix was used to evaluate the classification capability of the Random Forest model. It demonstrates how effectively the model differentiates between stress and non-stress conditions.

Table 7.2 Confusion Matrix Values

Actual / Predicted	Non-Stress	Stress
Non-Stress	245	5
Stress	6	220

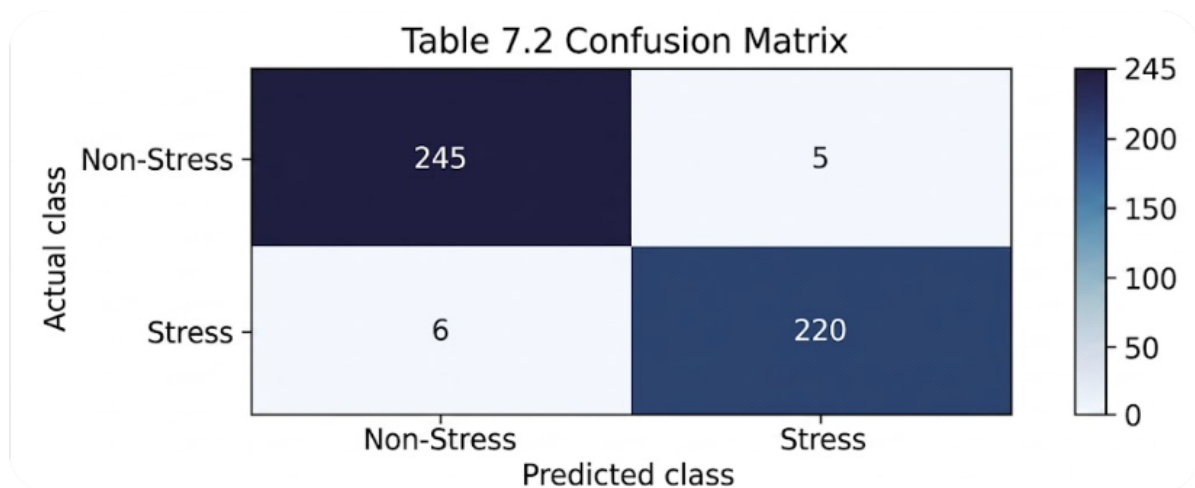


Fig 7.2 Confusion Matrix

The confusion matrix indicates that the proposed system correctly classified most stress and non-stress samples with very few misclassifications.

7.1.4 Signal Contribution Analysis

An ablation study was performed to analyze the contribution of individual signals:

- ACC signal achieved high standalone accuracy because motion data helps distinguish physical activity from stress.
- EDA provided strong emotional arousal information.
- BVP contributed cardiovascular response information.
- The combined multimodal approach produced the best results.

This confirms that combining physiological and motion signals improves robustness and context awareness in stress detection systems.

7.1.5 Federated Learning Results

Federated Learning was implemented to preserve user privacy during model training. Instead of sending raw physiological data to a centralized server, only local model parameters were shared.

Key observations include:

- Improved privacy protection
- Reduced centralized data dependency
- Better scalability for distributed healthcare systems
- Comparable performance with centralized learning approaches

The federated learning architecture demonstrated the feasibility of privacy-preserving healthcare monitoring systems.

7.2 Conclusion

This project successfully developed a real-time stress detection and monitoring system using IoT sensors, machine learning, and Federated Learning techniques. The proposed system effectively integrates physiological signals and motion data to perform accurate and context-aware stress analysis.

The ESP32 microcontroller, MAX30100/MAX30102 sensor, and MPU6050 motion sensor enabled efficient real-time data acquisition and wireless communication. The use of multimodal data significantly improved classification performance by distinguishing stress-induced physiological changes from physical activity-induced changes.

Different machine learning algorithms were evaluated, and the Random Forest classifier achieved the best performance with an accuracy of approximately 97.68%. The confusion matrix and evaluation metrics confirmed the robustness and reliability of the proposed model.

The incorporation of Federated Learning addressed one of the major limitations of traditional healthcare monitoring systems by preserving user privacy. Since raw data remains on local devices, the system provides a secure and scalable healthcare solution suitable for real-world deployment.

Advantages of the Proposed System

- Real-time stress monitoring
- High detection accuracy
- Privacy-preserving distributed learning
- Context-aware analysis using motion data
- Low-cost and portable implementation
- Scalable healthcare monitoring solution

7.2.1 Future Scope

The proposed work can be further enhanced by:

- Integrating deep learning architectures such as CNN-LSTM for improved temporal analysis
- Deploying the system on wearable healthcare devices
- Implementing cloud-based real-time visualization dashboards
- Expanding the dataset for improved generalization
- Incorporating additional physiological parameters such as ECG and temperature
- Developing mobile healthcare applications for remote patient monitoring

Overall, the proposed stress detection system provides an efficient, intelligent, and privacy-preserving healthcare solution capable of supporting future smart healthcare applications.

Code

```
# WESAD Dataset Understanding
## Objective
Build a subject-generalized machine learning model to classify stress using physiological
signals from WESAD.
## Signals Used
- EDA: Electrodermal Activity
- BVP: Blood Volume Pulse
- ACC: Accelerometer / motion
## Dataset Files
- Sx.pkl: synchronized data
- Sx_respiBAN.txt: chest sensor data
- Sx_E4_Data/: wrist sensor raw data
- Sx_quest.csv: questionnaire / label timing
## Labels
- 1: baseline
- 2: stress
- 3: amusement
- Ignore: 0, 5, 6, 7
## Important Design Choice
## Use subject-wise split to avoid data leakage and test true generalization.
```

Code :

```
import pickle
from pathlib import Path

subject_file = Path("/kaggle/input/datasets/orville/wesad-wearable-stress-affect-detection-
dataset/WESAD/S2/S2.pkl")

with open(subject_file, "rb") as f:
    data = pickle.load(f, encoding='latin1')

print(type(data))
print(data.keys() if hasattr(data, "keys") else "No keys found")

for k, v in data.items():
    try:
        print(f'{k}: shape = {v.shape}, type = {type(v)}')
    except Exception:
        print(f'{k}: type = {type(v)}')
signal_data = data['signal']

print(signal_data.keys())

for k, v in signal_data.items():
    try:
        print(f'{k}: shape = {v.shape}, type = {type(v)}')
```

```

except:
    print(f'{k}: type = {type(v)}')

wrist = signal_data['wrist']
chest = signal_data['chest']

print("WRIST KEYS:", wrist.keys())
for k, v in wrist.items():
    print(f'{k}: shape = {v.shape}')

print("\nCHEST KEYS:", chest.keys())
for k, v in chest.items():
    print(f'{k}: shape = {v.shape}')

```

output

Wrist signals:

EDA: (24,316, 1) → ~4 Hz BVP: (389,056, 1) → ~64 Hz ACC: (194,528, 3) → ~32 Hz

These are not aligned in time.

Labels:

label: (4,255,300,) → ~700 Hz (chest sampling rate)

Data Loading and Signal Alignment

We load all subjects from WESAD and align wrist signals (EDA, BVP, ACC) to a common sampling rate (32 Hz).

This naturally matches the sampling frequency of the ACC sensor, provides reasonable downsample for BVP

2 valid startergies

Option A

- Downsample everything to 4 Hz
Pros: clean, stable, low noise Cons: lose high-frequency info from BVP

Option B

- Upsample everything to 32 Hz (NOT 64 Hz)
Why 32 Hz?

Matches ACC naturally Preserves decent BVP resolution (downsample 64 → 32) Doesn't blow up data size like 64 Hz

Code:

```

import pickle
import numpy as np
import pandas as pd
from pathlib import Path
from scipy.signal import resample

```

```

def resample_to_length(signal, target_len):
    return resample(signal, target_len)

def load_subject(file_path):
    with open(file_path, "rb") as f:
        data = pickle.load(f, encoding='latin1')

    wrist = data['signal']['wrist']
    label = data['label']

    eda = wrist['EDA'].flatten()    # 4 Hz
    bvp = wrist['BVP'].flatten()    # 64 Hz
    acc = wrist['ACC']              # 32 Hz
    # Target = ACC length (32 Hz reference)
    target_len = len(acc)

    # Resample signals
    eda_resampled = resample_to_length(eda, target_len) # upsample
    bvp_resampled = resample_to_length(bvp, target_len) # downsample

    # Labels → from ~700 Hz → 32 Hz
    label_resampled = resample_to_length(label, target_len)
    label_resampled = np.round(label_resampled).astype(int)

    # Build DataFrame
    df = pd.DataFrame({
        'EDA': eda_resampled,
        'BVP': bvp_resampled,
        'ACC_X': acc[:, 0],
        'ACC_Y': acc[:, 1],
        'ACC_Z': acc[:, 2],
        'label': label_resampled
    })
    return df

def load_all_subjects(data_dir):
    data = []

    for subject_path in sorted(Path(data_dir).glob("S*/S*.pkl")):
        print(f'Loading {subject_path.name}')
        df = load_subject(subject_path)
        df['subject'] = subject_path.parent.name
        data.append(df)

    full_df = pd.concat(data, ignore_index=True)
    return full_df

data_dir = "/kaggle/input/datasets/orvile/wesad-wearable-stress-affect-detection-dataset/
WESAD"

```

```
df = load_all_subjects(data_dir)
```

```
print(df.shape)
```

```
df.head()
```

```
print(df['label'].value_counts())
```

```
print(df.isnull().sum())
```

```
print(df.describe())
```

* Data Shape: ~2.77 million rows → expected after resampling to 32 Hz across subjects

* No missing values

* Fourier resampling of labels

0	→	1,264,118	✗	ignore
1	→	563,618	✓	baseline
2	→	318,944	✓	stress
3	→	178,426	✓	amusement
4,5,6,7,8,-1			✗	Not needed

Output:

Cleaning of Labels

1. Keep only valid labels → 1, 2, 3
2. Drop everything else
3. Convert to binary classification a. Stress (2) → 1 b. Baseline (1) + Amusement (3) → 0

Code

```
valid_labels = [1, 2, 3]
```

```
df = df[df['label'].isin(valid_labels)].copy()
```

Convert to binary:

Stress = 1, Non-stress = 0

```
df['target'] = df['label'].apply(lambda x: 1 if x == 2 else 0)
```

```
print(df['label'].value_counts())
```

```
print(df['target'].value_counts())
```

output:

Windowing

Each row = one timestamp

ML models need: fixed-size chunks with features

Design Decision

Window size: 30 seconds Sampling rate: 32 Hz

window_size = 30 * 32 = 960 samples Overlapping (50%) -> 480

Window labeling strategy

Majority label per 30s window size

Code:

```
def create_windows(df, window_size=960, step=480):
    features = []
    labels = []
    subjects = []

    for subject in df['subject'].unique():
        subject_df = df[df['subject'] == subject]

        data = subject_df[['EDA', 'BVP', 'ACC_X', 'ACC_Y', 'ACC_Z']].values
        target = subject_df['target'].values

        for start in range(0, len(data) - window_size, step):
            end = start + window_size

            window_data = data[start:end]
            window_label = target[start:end]

            # Majority label
            label = np.bincount(window_label).argmax()

            features.append(window_data)
            labels.append(label)
            subjects.append(subject)

    return np.array(features), np.array(labels), np.array(subjects)

X, y, subjects = create_windows(df)

print(X.shape) # (num_windows, 960, 5)
print(y.shape)
```

Output:

- X: (2187, 960, 5) → correct windowed signals
- y: (2187,) → labels per window

Code:

Cannot directly train the ML model on these windows, ML model work best on 'Feature Vector'

Logistic Regression → expects 2D

Random Forest → expects tabular features

Output :

Feature Engineering

Features to extract for each signal

- EDA

- BVP
- ACC_X, ACC_Y, ACC_Z

Code:

```
def extract_features(X):
    feature_list = []

    for window in X:
        features = []

        for i in range(window.shape[1]): # 5 signals
            signal = window[:, i]

            features.extend([
                np.mean(signal),
                np.std(signal),
                np.min(signal),
                np.max(signal),
                np.median(signal),
                np.max(signal) - np.min(signal)
            ])

        feature_list.append(features)

    return np.array(feature_list)
X_features = extract_features(X)

print(X_features.shape)
import pandas as pd
```

```
feature_df = pd.DataFrame(X_features)
print(feature_df.describe())
```

Why EDA, BVP, and ACC are used

We select three physiological modalities:

- **EDA (Electrodermal Activity)**
Reflects sympathetic nervous system activation → strong indicator of stress
 - **BVP (Blood Volume Pulse)**
Captures cardiovascular response → stress affects heart rate and variability
 - **ACC (Accelerometer)**
Captures body movement → helps distinguish stress from physical activity
- Signal Complementarity
- EDA → emotional arousal
 - BVP → physiological response
 - ACC → motion context

Together, they provide a **holistic representation of stress vs activity**

Code:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupShuffleSplit
```

```
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
```

```
train_idx, test_idx = next(gss.split(X_features, y, groups=subjects))
```

```
X_train, X_test = X_features[train_idx], X_features[test_idx]
```

```
y_train, y_test = y[train_idx], y[test_idx]
```

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
model.fit(X_train, y_train)
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
importances = model.feature_importances_
```

```
# Create feature names
```

```
signals = ['EDA', 'BVP', 'ACC_X', 'ACC_Y', 'ACC_Z']
```

```
stats = ['mean', 'std', 'min', 'max', 'median', 'range']
```

```
feature_names = [
```

```
    f'{sig}_{stat}' for sig in signals for stat in stats ]
```

```
plt.figure(figsize=(12, 6))
```

```
plt.barh(feature_names, importances)
```

```
plt.title("Feature Importance (Random Forest)")
```

```
plt.xlabel("Importance Score")
```

```
plt.ylabel("Features")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
signal_importance = {}
```

```
for i, name in enumerate(feature_names):
```

```
    signal = name.split('_')[0]
```

```
    signal_importance.setdefault(signal, 0)
```

```
    signal_importance[signal] += importances[i]
```

```
print(signal_importance)
```

```
plt.bar(signal_importance.keys(), signal_importance.values())
```

```
plt.title("Signal-Level Importance")
```

```
plt.xlabel("Signal")
```

```
plt.ylabel("Total Importance")
```

```
plt.show()
```

Feature Importance Analysis

To validate the selection of physiological signals (EDA, BVP, ACC), we trained a Random Forest model using subject-independent splits.

This ensures that importance reflects generalizable patterns rather than subject-specific artifacts.

Key Findings

- EDA shows strong importance → reflects stress-related arousal
- BVP contributes to cardiovascular response modeling
- ACC helps differentiate physical movement from stress

Together, these signals provide complementary information for robust classification.

Conclusion

EDA, BVP, and ACC together provide complementary information:

- ACC → captures physical activity
- EDA → captures emotional arousal
- BVP → captures cardiovascular response

A combination of all three is necessary for a robust and generalizable stress detection system.

```

from sklearn.model_selection import GroupShuffleSplit
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

def train_and_evaluate(X, y, subjects):
    gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
    train_idx, test_idx = next(gss.split(X, y, groups=subjects))

    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

    preds = model.predict(X_test)

    acc = accuracy_score(y_test, preds)

    return acc, y_test, preds

# Feature slices
X_eda = X_features[:, 0:6]
X_bvp = X_features[:, 6:12]
X_acc = X_features[:, 12:30]
X_all = X_features

results = {}

# EDA only
acc_eda, y_test, preds = train_and_evaluate(X_eda, y, subjects)
results['EDA'] = acc_eda

# BVP only
acc_bvp, _, _ = train_and_evaluate(X_bvp, y, subjects)
results['BVP'] = acc_bvp

# ACC only

```

```
acc_acc, _, _ = train_and_evaluate(X_acc, y, subjects)
results['ACC'] = acc_acc

# All signals
acc_all, _, _ = train_and_evaluate(X_all, y, subjects)
results['ALL'] = acc_all
```

```
print(results)
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(6, 4))
plt.bar(results.keys(), results.values())
```

```
plt.title("Ablation Study: Signal Contribution")
plt.xlabel("Signal")
plt.ylabel("Accuracy")
```

```
for i, v in enumerate(results.values()):
    plt.text(i, v + 0.005, f'{v:.3f}', ha='center')
```

```
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

Key Insight

- ACC achieves the highest standalone accuracy, indicating strong motion-based separability.
- EDA performs moderately, capturing stress-related physiological changes.
- BVP alone performs weaker, suggesting it acts as a supporting signal.

The combined model achieves the best performance, demonstrating that:

- Motion (ACC) + physiological signals (EDA, BVP) together improve robustness. This validates the selection of multimodal features.

Model Building and Evaluation

We train three models:

1. Logistic Regression (baseline)
2. Random Forest (ensemble)
3. Neural Network (MLP)

Evaluation is done using subject-wise split to ensure generalization.

```
from sklearn.model_selection import GroupShuffleSplit
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(X_features, y, groups=subjects))
```

```
X_train, X_test = X_features[train_idx], X_features[test_idx]
y_train, y_test = y[train_idx], y[test_idx]
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
```

```

def evaluate_model(name, model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    acc = accuracy_score(y_test, preds)
    prec = precision_score(y_test, preds)
    rec = recall_score(y_test, preds)
    f1 = f1_score(y_test, preds)

    print(f"\n{name}")
    print("Accuracy:", round(acc, 4))
    print("Precision:", round(prec, 4))
    print("Recall:", round(rec, 4))
    print("F1-score:", round(f1, 4))

return {
    "model": name,
    "accuracy": acc,
    "precision": prec,
    "recall": rec,
    "f1": f1
}

def plot_conf_matrix(model, X_test, y_test, title):
    from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)

    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Non-Stress", "Stress"]
    )

    disp.plot(cmap="Blues")
    plt.title(title)
    plt.show()

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

log_model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000))
])

res_log = evaluate_model("Logistic Regression", log_model, X_train, X_test, y_train, y_test)
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy="most_frequent")

```

```

res_dummy = evaluate_model(
    "Dummy Classifier",
    dummy,
    X_train,
    X_test,
    y_train,
    y_test
)

from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)

res_dt = evaluate_model(
    "Decision Tree",
    dt_model,
    X_train,
    X_test,
    y_train,
    y_test
)
dt_model.fit(X_train, y_train)

preds = dt_model.predict(X_test)
signals = ['EDA', 'BVP', 'ACC_X', 'ACC_Y', 'ACC_Z']
stats = ['mean', 'std', 'min', 'max', 'median', 'range']

feature_names = [f"{sig}_{stat}" for sig in signals for stat in stats]

import matplotlib.pyplot as plt
from sklearn import tree

plt.figure(figsize=(20, 12))

tree.plot_tree(
    dt_model,
    feature_names=feature_names,
    class_names=['Non-Stress', 'Stress'],
    filled=True,
    max_depth=3 # keep it readable
)

plt.title("Decision Tree Visualization (Top Levels)")
plt.show()

plot_conf_matrix(dt_model, X_test, y_test, "Decision Tree")
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=300,

```

```

    class_weight='balanced',
    random_state=42
)

res_rf = evaluate_model("Random Forest", rf_model, X_train, X_test, y_train, y_test)
rf_model.fit(X_train, y_train)

preds = rf_model.predict(X_test)
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "n_estimators": [200, 300, 500],
    "max_depth": [10, 20, 30, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ['sqrt', 'log2']
}

rf = RandomForestClassifier(class_weight='balanced', random_state=42)

search = RandomizedSearchCV(
    rf,
    param_distributions=param_dist,
    n_iter=20,
    scoring='f1',
    cv=3,
    n_jobs=-1,
    random_state=42
)

search.fit(X_train, y_train)

best_rf = search.best_estimator_
res_rf_tuned = evaluate_model(
    "Random Forest (Tuned)",
    best_rf,
    X_train,
    X_test,
    y_train,
    y_test
)
rf_model.fit(X_train, y_train)

preds = rf_model.predict(X_test)
plot_conf_matrix(rf_model, X_test, y_test, "Decision Tree")
from sklearn.neural_network import MLPClassifier

mlp_model = Pipeline([
    ("scaler", StandardScaler()),

```

```
("mlp", MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=300, random_state=42))
])
```

```
res_mlp = evaluate_model("Neural Network (MLP)", mlp_model, X_train, X_test, y_train,
y_test)
```

```
import pandas as pd
```

```
results_df = pd.DataFrame([res_dummy, res_dt, res_rf_tuned, res_mlp])
results_df
```

```
import matplotlib.pyplot as plt
```

```
results_df.set_index("model")[["accuracy", "f1"]].plot(kind="bar", figsize=(8,5))
```

```
plt.title("Model Comparison")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

Model Comparison Insights

- Decision Tree provides a simple baseline but may underfit complex patterns.
 - Random Forest performs strongly due to its ability to capture non-linear relationships.
 - The Neural Network captures more complex interactions but may require tuning.
- Overall, ensemble methods like Random Forest provide the best balance of performance and stability for this task.

This Python 3 environment comes with many helpful analytics libraries installed
It is defined by the kaggle/python Docker image: <https://github.com/kaggle/docker-python>
For example, here's several helpful packages to load

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

Input data files are available in the read-only "../input/" directory
For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

```
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

Step 0: Imports

```
import pandas as pd
```

[illegible]

```

0  2142701
1   800800
4   537599
2   430500
3   253400
6    45500
7    44800
Name: count, dtype: int64
# Example: check the available sensors
print(subject_data['signal'].keys())
print(subject_data['signal']['chest'].keys())
print(subject_data['signal']['wrist'].keys())

# Example: look at ECG values
ecg_data = subject_data['signal']['chest']['ECG']
print(ecg_data.shape)
print(ecg_data[:10]) #first 10 samples
dict_keys(['chest', 'wrist'])
dict_keys(['ACC', 'ECG', 'EMG', 'EDA', 'Temp', 'Resp'])
dict_keys(['ACC', 'BVP', 'EDA', 'TEMP'])
(4255300, 1)
[[0.02142334]
 [0.02032471]
 [0.01652527]
 [0.01670837]
 [0.01167297]
 [0.00489807]
 [0.00279236]
 [0.00608826]
 [0.00993347]
 [0.01039124]]
# Example: check the available sensors
print(subject_data['signal'].keys())

# Example: look at ECG values
ecg_chest_data = subject_data['signal']['chest']['ECG']
print(ecg_chest_data.shape)
print(ecg_chest_data[:10])
#first 10 samples
dict_keys(['chest', 'wrist'])
(4255300, 1)
[[0.02142334]
 [0.02032471]
 [0.01652527]
 [0.01670837]
 [0.01167297]
 [0.00489807]
 [0.00279236]

```

```

[0.00608826]
[0.00993347]
[0.01039124]]
acc_chest_data = subject_data['signal']['chest']['ACC']
acc_chest_data
array([[ 0.95539999, -0.222    , -0.55799997],
       [ 0.92579997, -0.2216   , -0.55379999],
       [ 0.90820003, -0.21960002, -0.53920001],
       ...,
       [ 0.87179995, -0.12379998, -0.30419999],
       [ 0.87300003, -0.12339997, -0.30260003],
       [ 0.87020004, -0.12199998, -0.30220002]])
labels = subject_data['label']
print(labels.shape)
print(np.unique(labels)) # shows 0,1,2,... for different states
(4255300,)
[0 1 2 3 4 6 7]
labels = subject_data['label']
labels_series = pd.Series(labels)

# Count of each label
counts = labels_series.value_counts().sort_index()
print(counts)
0    2142701
1     800800
2     430500
3     253400
4     537599
6      45500
7      44800
Name: count, dtype: int64
import numpy as np

# Original labels
labels = subject_data['label']

# Identify indices to keep (ignore 5, 6, 7)
keep_indices = np.isin(labels, [0,1,2,3,4])

# Filter labels
labels_clean = labels[keep_indices]

# Filter ECG (or any other sensor) correspondingly
ecg_clean = ecg_data[keep_indices]

# Check shapes
print("Filtered ECG shape:", ecg_clean.shape)
print("Filtered labels shape:", labels_clean.shape)

```

```

# Check unique labels after filtering
print("Unique labels:", np.unique(labels_clean))
Filtered ECG shape: (4165000, 1)
Filtered labels shape: (4165000,)
Unique labels: [0 1 2 3 4]
# ECG data for the subject
ecg_data = subject_data['signal']['chest']['ECG']

# Plot first 5000 samples (for visibility)
plt.figure(figsize=(15,4))
plt.plot(ecg_data[:5000])
plt.title("ECG Signal (First 5000 Samples)")
plt.xlabel("Time (samples)")
plt.ylabel("ECG amplitude")
plt.show()

```

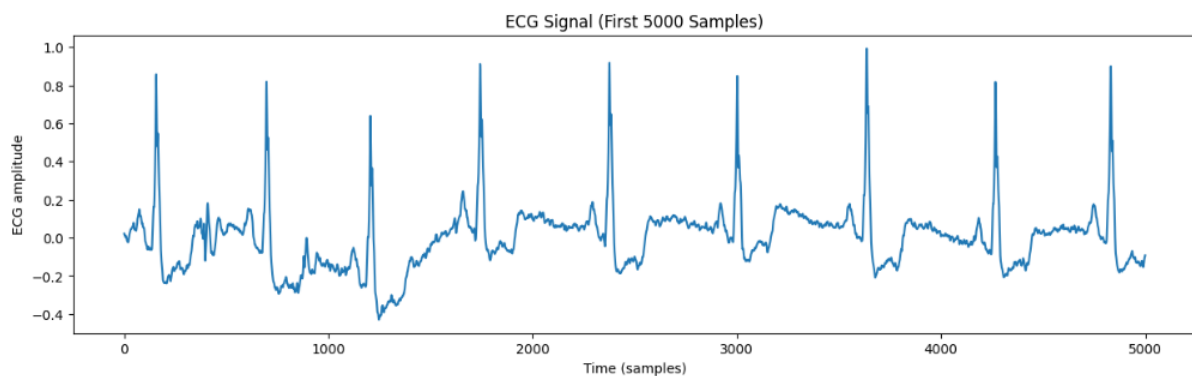


Fig 7.3 ECG signal result

```

# Convert to pandas Series
labels_series = pd.Series(labels_clean)

# Count of each label
counts = labels_series.value_counts().sort_index()

# Plot bar chart
plt.figure(figsize=(8,5))
plt.bar(counts.index, counts.values, color='skyblue')
plt.xlabel("Label")
plt.ylabel("Count")
plt.title("Distribution of Labels (0-4)")
plt.show()

```

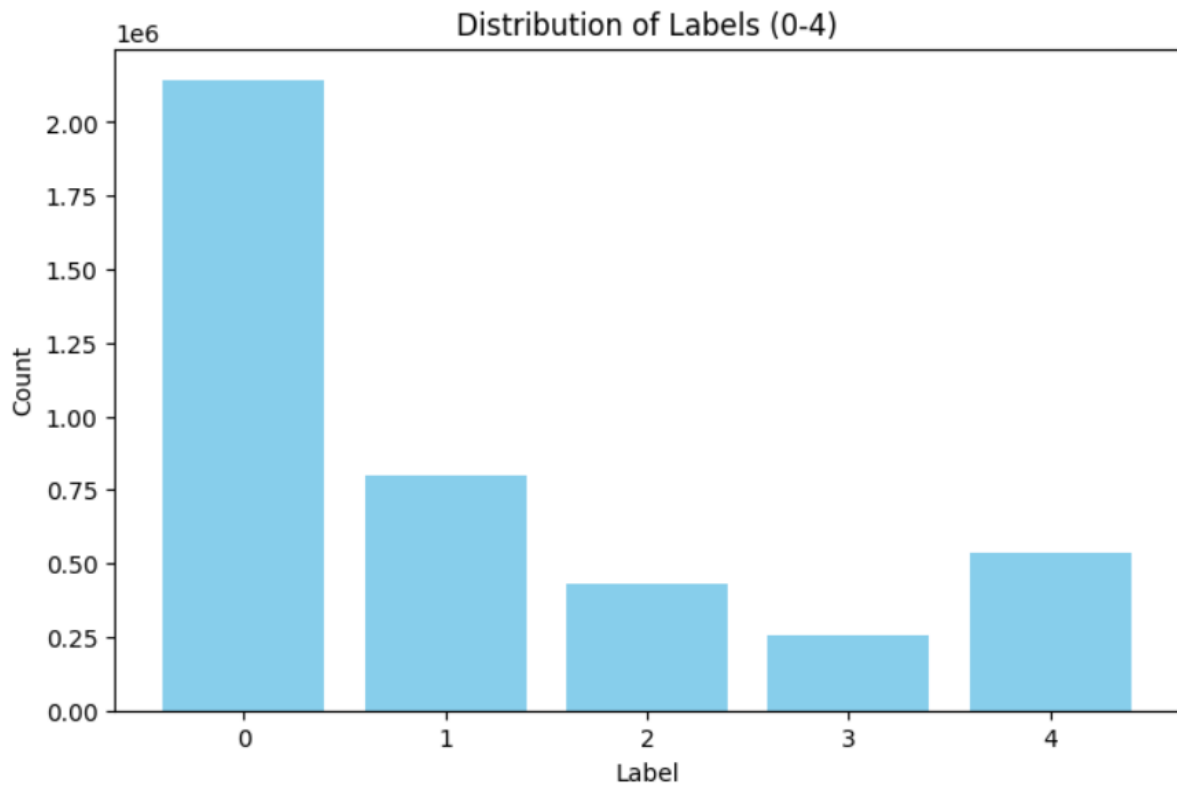


Fig 7.4 Distribution of WESAD Dataset Labels

```
# Make sure ECG is 1-D
ecg_signal = ecg_clean

# If it's 2-D, flatten it
if ecg_signal.ndim > 1:
    ecg_signal = ecg_signal[:,0] # take first channel

print("ECG shape after flattening:", ecg_signal.shape)
ECG shape after flattening: (4165000,)
from scipy.signal import find_peaks
peaks, _ = find_peaks(ecg_signal, distance=200)

print("Number of detected peaks:", len(peaks))
Number of detected peaks: 16348
plt.figure(figsize=(15,4))
plt.plot(ecg_signal[:5000], label='ECG')
plt.plot(peaks[peaks<5000], ecg_signal[peaks[peaks<5000]], 'rx', label='R-peaks')
plt.title("ECG with R-peaks")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.legend()
plt.show()
```

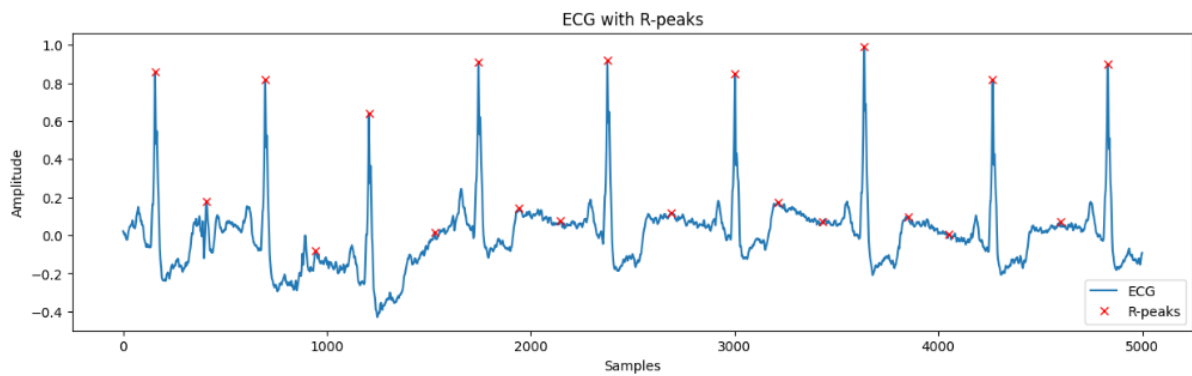


Figure 7.5 ECG Signal with Detected R-Peaks

```
# Assuming sampling rate = 700 Hz for chest ECG
fs = 700
```

```
# RR intervals in seconds
```

```
rr_intervals = np.diff(peaks) / fs
```

```
print("First 10 RR intervals (s):", rr_intervals[:10])
```

```
First 10 RR intervals (s): [0.36      0.41142857 0.35142857 0.37571429 0.46      0.30714286
 0.28571429 0.28714286 0.33142857 0.44857143]
```

```
mean_rr = np.mean(rr_intervals)
```

```
std_rr = np.std(rr_intervals)
```

```
heart_rate = 60 / mean_rr # bpm
```

```
hrv = std_rr # simple HRV measure (SDNN)
```

```
print(f"Mean RR: {mean_rr:.3f}s, Heart Rate: {heart_rate:.2f} bpm, HRV: {hrv:.3f}")
```

```
Mean RR: 0.364s, Heart Rate: 164.85 bpm, HRV: 0.093
```

```
window_size = 5 * fs # 5 sec
```

```
step_size = window_size // 2 # 50% overlap
```

```
features = []
```

```
window_labels = []
```

```
for start in range(0, len(ecg_clean) - window_size, step_size):
```

```
    end = start + window_size
```

```
    window_signal = ecg_clean[start:end]
```

```
    # If it's 2-D, flatten it
```

```
    if window_signal.ndim > 1:
```

```
        window_signal = window_signal[:,0] # take first channel
```

```
    # R-peak detection
```

```
    peaks, _ = find_peaks(window_signal, distance=200)
```

```
    # Skip windows with too few peaks
```

```
    if len(peaks) < 2:
```

```
        continue
```

```
    rr_intervals = np.diff(peaks) / fs
```

```

# Features
mean_rr = np.mean(rr_intervals)
std_rr = np.std(rr_intervals)
heart_rate = 60 / mean_rr
hrv = std_rr

features.append([mean_rr, std_rr, heart_rate, hrv])

# Label for this window (most frequent label in the segment)
window_label = labels_clean[start:end]
counts = np.bincount(window_label)
window_labels.append(np.argmax(counts))

# Convert to numpy arrays
ecg_features = np.array(features)
ecg_window_labels = np.array(window_labels)

print("Features shape:", ecg_features.shape)

print("Window labels shape:", ecg_window_labels.shape)

Features shape: (2378, 4)
Window labels shape: (2378,)

```

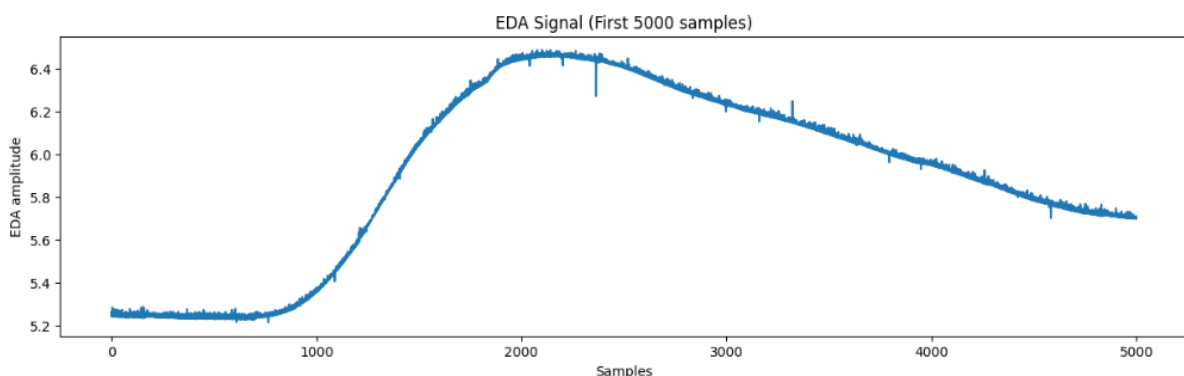


Fig 7.6.EDA signal — first 5000 samples.

```

print("EDA Mean:", np.mean(eda_signal))
print("EDA Std:", np.std(eda_signal))
print("EDA Min/Max:", np.min(eda_signal), np.max(eda_signal))

EDA Mean: 1.5283874467075147
EDA Std: 1.238424840251518
EDA Min/Max: 0.263214111328125 7.576751708984375
print(eda_signal.shape)

(4255300, 1)
# Match label length to EDA signal length
labels_resampled = np.repeat(labels_clean, len(eda_signal) // len(labels_clean) + 1)
labels_resampled = labels_resampled[:len(eda_signal)]

from scipy.signal import find_peaks

window_size = 5 * fs # same as ECG

```

```

step_size = window_size // 2

eda_features = []
eda_window_labels = []

for start in range(0, len(eda_signal) - window_size, step_size):
    end = start + window_size
    window = eda_signal[start:end]

    # If it's 2-D, flatten it
    if window.ndim > 1:
        window = window[:,0] # take first channel

    # Features
    mean_val = np.mean(window)
    std_val = np.std(window)
    max_val = np.max(window)
    min_val = np.min(window)

    peaks, _ = find_peaks(window, height=np.mean(window))
    num_peaks = len(peaks)

    eda_features.append([mean_val, std_val, max_val, min_val, num_peaks])

    # Label for this window
    window_label = labels_resampled[start:end]
    if len(window_label) == 0:
        continue
    counts = np.bincount(window_label)
    eda_window_labels.append(np.argmax(counts))

# Convert to numpy arrays
eda_features = np.array(eda_features)
eda_window_labels = np.array(eda_window_labels)

print("EDA features shape:", eda_features.shape)
print("EDA window labels shape:", eda_window_labels.shape)
np.unique(eda_window_labels, return_counts=True)

EDA features shape: (2430, 5)
EDA window labels shape: (2430,)
(array([0, 1, 2]), array([1023, 915, 492]))
acc_signal = subject_data['signal']['chest']['ACC'] # shape: (n_samples, 3)
print("ACC shape:", acc_signal.shape)

plt.figure(figsize=(15,4))
plt.plot(acc_signal[:2000, 0], label='X')
plt.plot(acc_signal[:2000, 1], label='Y')
plt.plot(acc_signal[:2000, 2], label='Z')
plt.title("Accelerometer (First 2000 samples)")
plt.xlabel("Samples")

```

```
plt.ylabel("Acceleration (g)")
plt.legend()
plt.show()
```

ACC shape: (4255300, 3)

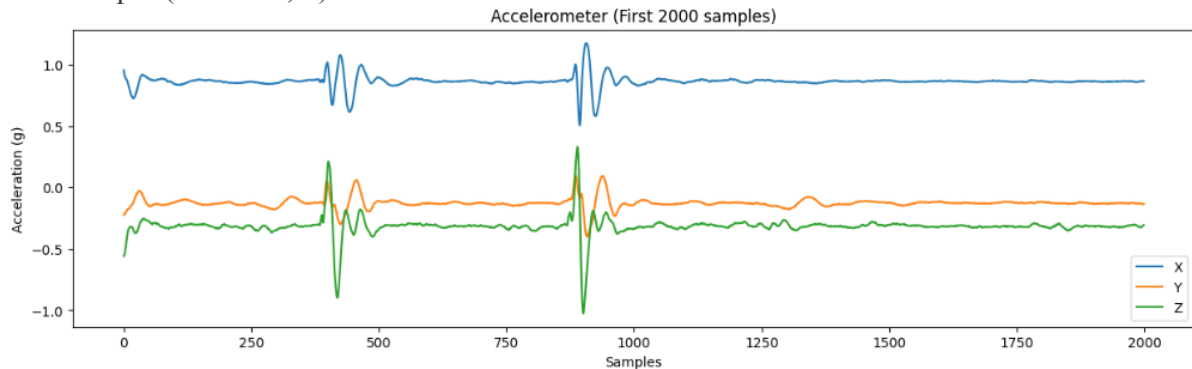


Fig 7.7 .Accelerometer signal — first 2000 samples.

```
acc_magnitude = np.sqrt(np.sum(acc_signal**2, axis=1))
```

```
plt.figure(figsize=(15,4))
plt.plot(acc_magnitude[:2000])
plt.title("ACC Magnitude (First 2000 samples)")
plt.xlabel("Samples")
plt.ylabel("Magnitude (g)")
plt.show()
```

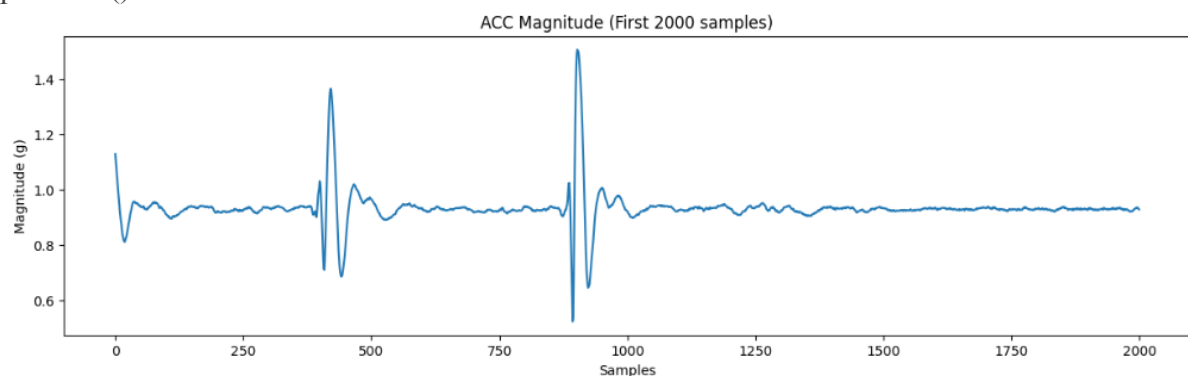


Fig 7.8 .ACC Magnitude signal — first 2000 samples.

```
acc_features = []
acc_window_labels = []

# Resample labels to match ACC signal length
labels_resampled = np.repeat(labels_clean, len(acc_magnitude) // len(labels_clean) + 1)
labels_resampled = labels_resampled[:len(acc_magnitude)]
```

```
for start in range(0, len(acc_magnitude) - window_size, step_size):
    end = start + window_size
    window = acc_magnitude[start:end]
```

```
# Skip empty windows
if len(window) == 0:
    continue
```

```

# Basic statistical features
mean_val = np.mean(window)
std_val = np.std(window)
max_val = np.max(window)
min_val = np.min(window)
rms = np.sqrt(np.mean(window**2)) # root mean square

acc_features.append([mean_val, std_val, max_val, min_val, rms])

# Label for this window
window_label = labels_resampled[start:end]
counts = np.bincount(window_label)
acc_window_labels.append(np.argmax(counts))

acc_features = np.array(acc_features)
acc_window_labels = np.array(acc_window_labels)

print("ACC features shape:", acc_features.shape)
print("ACC window labels shape:", acc_window_labels.shape)
ACC features shape: (2430, 5)
ACC window labels shape: (2430,)
unique, counts = np.unique(acc_window_labels, return_counts=True)
print(dict(zip(unique, counts)))
{0: 1023, 1: 915, 2: 492}
temp_signal = subject_data['signal']['chest']['Temp']
print("TEMP shape:", temp_signal.shape)

plt.figure(figsize=(15,4))
plt.plot(temp_signal[:5000])
plt.title("Temperature Signal (First 5000 samples)")
plt.xlabel("Samples")
plt.ylabel("Temperature (°C)")
plt.show()
TEMP shape: (4255300, 1)

```

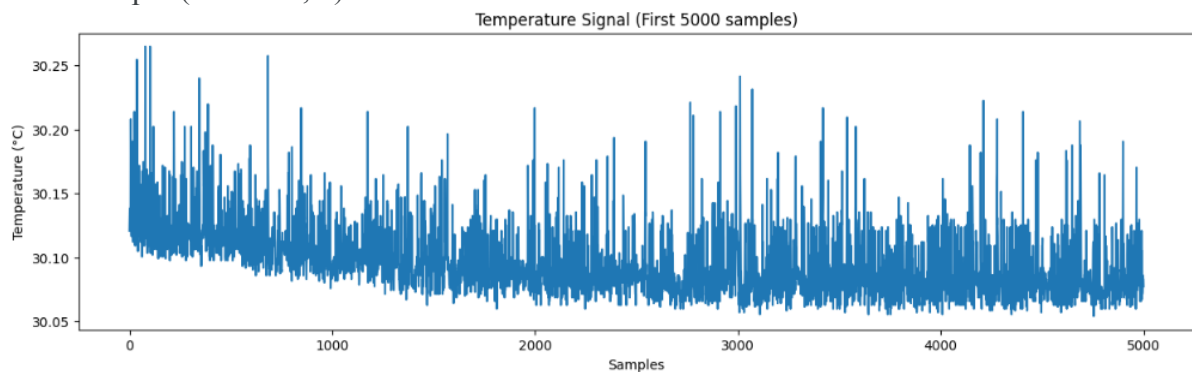


Fig 7.9 Temperature signal — first 5000 samples.

```

labels_resampled = np.repeat(labels_clean, len(temp_signal) // len(labels_clean) + 1)
labels_resampled = labels_resampled[:len(temp_signal)]

```

```

temp_features = []
temp_window_labels = []

for start in range(0, len(temp_signal) - window_size, step_size):
    end = start + window_size
    window = temp_signal[start:end]

    # Skip empty or NaN windows
    if len(window) == 0 or np.isnan(window).any():
        continue

    # Compute scalar features safely
    mean_val = float(np.mean(window))
    std_val = float(np.std(window))
    max_val = float(np.max(window))
    min_val = float(np.min(window))
    slope = float((window[-1] - window[0]) / len(window))

    temp_features.append([mean_val, std_val, max_val, min_val, slope])

    window_label = labels_resampled[start:end]
    if len(window_label) == 0:
        continue
    counts = np.bincount(window_label)
    temp_window_labels.append(np.argmax(counts))

# Convert to numpy arrays
temp_features = np.array(temp_features, dtype=float)
temp_window_labels = np.array(temp_window_labels, dtype=int)

print("TEMP features shape:", temp_features.shape)
print("TEMP window labels shape:", temp_window_labels.shape)
/tmp/ipykernel_13/1503991927.py:17: DeprecationWarning: Conversion of an array with
ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single
element from your array before performing this operation. (Deprecated NumPy 1.25.)
    slope = float((window[-1] - window[0]) / len(window))
TEMP features shape: (2430, 5)
TEMP window labels shape: (2430,)
unique, counts = np.unique(temp_window_labels, return_counts=True)
print(dict(zip(unique, counts)))
{0: 1023, 1: 915, 2: 492}
print("ECG:", ecg_features.shape)
print("EDA:", eda_features.shape)
print("ACC:", acc_features.shape)
print("TEMP:", temp_features.shape)
ECG: (2378, 4)
EDA: (2430, 5)
ACC: (2430, 5)
TEMP: (2430, 5)

```

```

# Ensure all have the same number of windows
min_len = min(len(ecg_features), len(eda_features), len(acc_features), len(temp_features))
print("Trimming all to length:", min_len)

# Trim all to same size
ecg_f = ecg_features[:min_len]
eda_f = eda_features[:min_len]
acc_f = acc_features[:min_len]
temp_f = temp_features[:min_len]

# Combine horizontally (feature-wise)
final_features = np.hstack([ecg_f, eda_f, acc_f, temp_f])
final_labels = ecg_window_labels[:min_len] # labels are aligned per window

print("Final features shape:", final_features.shape)
print("Final labels shape:", final_labels.shape)

Trimming all to length: 2378
Final features shape: (2378, 19)
Final labels shape: (2378,)
df = pd.DataFrame(final_features)
df['label'] = final_labels

print(df['label'].value_counts())
df.head()
label
0    1223
1     458
4     306
2     246
3     145
Name: count, dtype: int64

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    final_features, final_labels, test_size=0.2, random_state=42, stratify=final_labels
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

Train shape: (1902, 19)
Test shape: (476, 19)
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    max_depth=5, # keep it shallow for interpretability
    random_state=42
)

```

```
dt.fit(X_train, y_train)
```

```
DecisionTreeClassifier
```

```
DecisionTreeClassifier(max_depth=5, random_state=42)
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
y_pred = dt.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['Baseline','Stress','Activity'],
```

```
            yticklabels=['Baseline','Stress','Activity'])
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Decision Tree - Confusion Matrix")
```

```
plt.show()
```

Accuracy: 0.8928571428571429

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.91	0.90	245
1	1.00	0.92	0.96	92
2	0.79	0.98	0.87	49
3	0.80	0.69	0.74	29
4	0.94	0.79	0.86	61
accuracy			0.89	476
macro avg	0.88	0.86	0.87	476
weighted avg	0.90	0.89	0.89	476

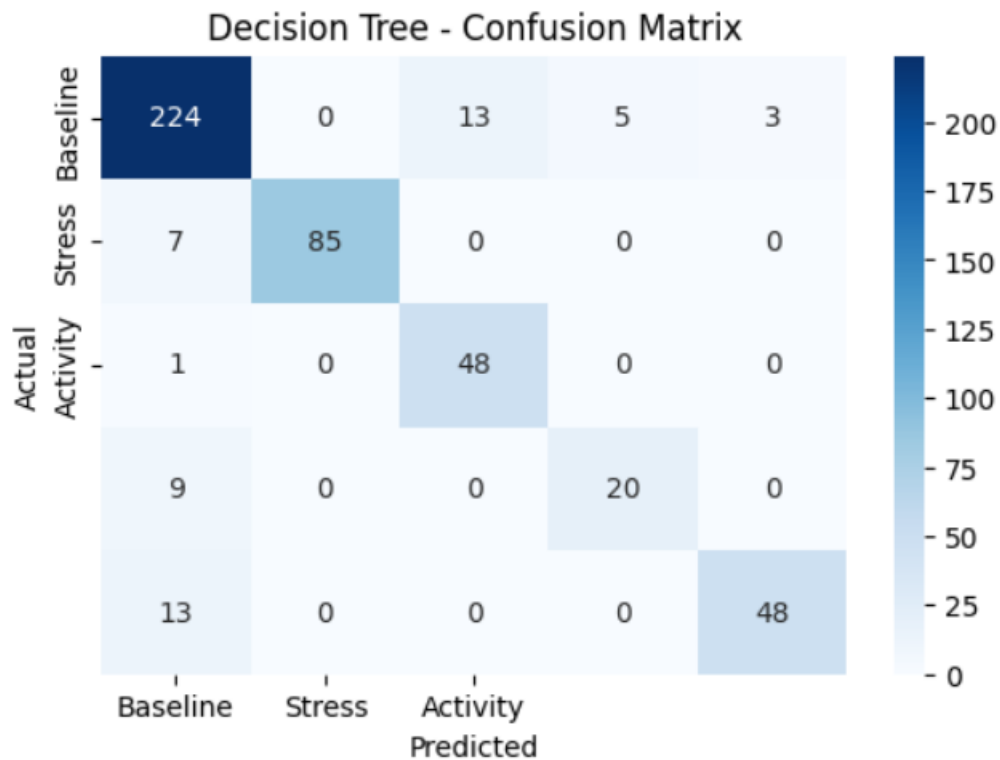
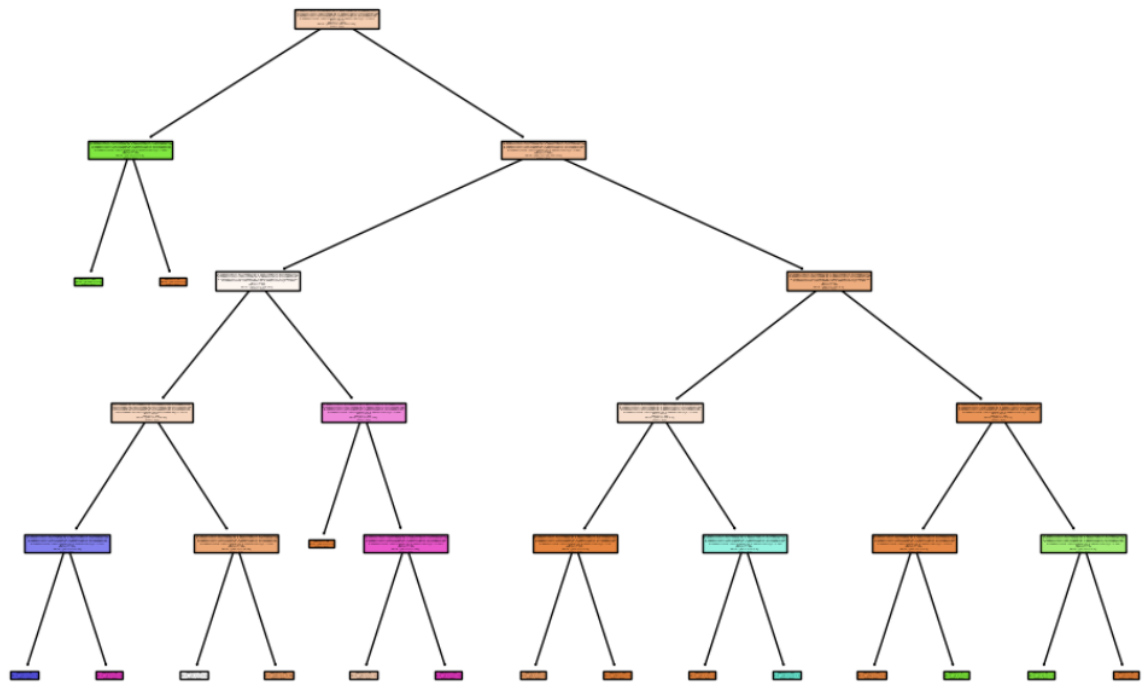


Fig 7.10 Decision Tree – Confusion Matrix

```
import matplotlib.pyplot as plt
from sklearn import tree
plt.rcParams.update({
    'figure.figsize': (20, 15),    # larger figure size
    'font.size': 14,               # bigger text
    'axes.titlesize': 16,         # title size
    'axes.labelsize': 14,        # axis label size
    'xtick.labelsize': 12,
    'ytick.labelsize': 12,
    'legend.fontsize': 12,
    'lines.linewidth': 1.5,
})

plt.figure(figsize=(12,8))
tree.plot_tree(dt, feature_names = final_features[:19] , class_names=True, filled=True)
plt.show()
```



```
X_train.shape
(1902, 19)
# top_features = feature_importance_df.sort_values(by='importance',
ascending=False).head(15)
# print(top_features)
from sklearn.dummy import DummyClassifier

dc = DummyClassifier(strategy="stratified")
# dc = DummyClassifier(strategy="most_frequent")
# dc = DummyClassifier(strategy="uniform")
# dc = DummyClassifier(strategy="constant", constant = 0)

dc.fit(X_train, y_train)

☒ DummyClassifier
DummyClassifier(strategy='stratified')

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

y_pred = dc.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Baseline','Stress','Activity'],
            yticklabels=['Baseline','Stress','Activity'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Accuracy: 0.32142857142857145

Classification Report:

	precision	recall	f1-score	support
0	0.53	0.49	0.51	245
1	0.20	0.18	0.19	92
2	0.09	0.10	0.10	49
3	0.07	0.10	0.09	29
4	0.11	0.11	0.11	61

accuracy		0.32	476
macro avg	0.20	0.20	0.20 476
weighted avg	0.34	0.32	0.33 476

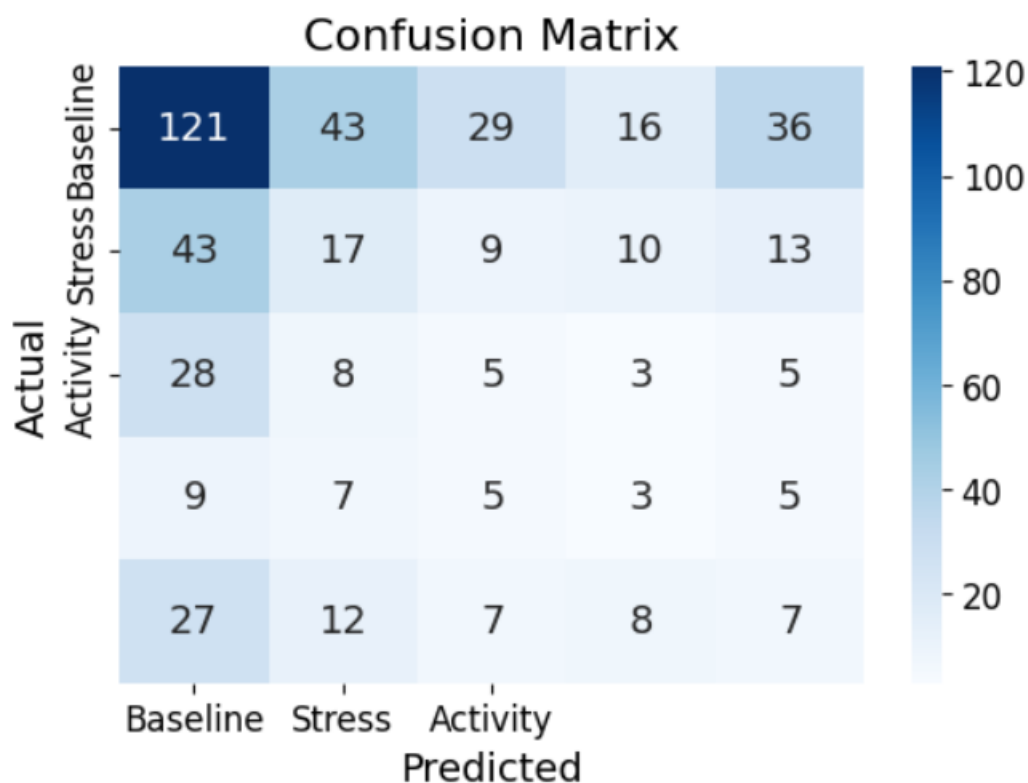


Fig 7.12 Confusion Matrix

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(
    n_estimators=200,
```

```

    max_depth=10,
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)

```

☑ RandomForestClassifier

```

RandomForestClassifier(max_depth=10, n_estimators=200, n_jobs=-1,
                        random_state=42)

```

```

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

y_pred = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Baseline','Stress','Activity'],
            yticklabels=['Baseline','Stress','Activity'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

Accuracy: 0.976890756302521

```

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	245
1	1.00	0.95	0.97	92
2	1.00	1.00	1.00	49
3	1.00	0.93	0.96	29
4	1.00	0.93	0.97	61
accuracy			0.98	476
macro avg	0.99	0.96	0.98	476
weighted avg	0.98	0.98	0.98	476

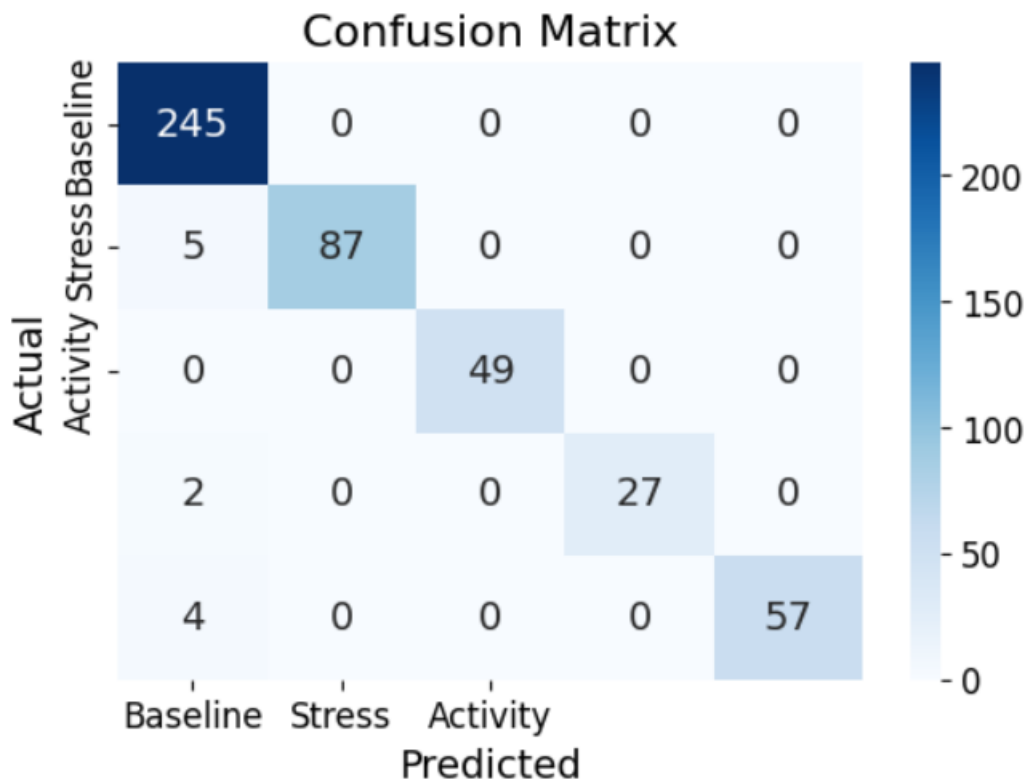


Fig 7.13 Random Forest – Confusion Matrix

```
import numpy as np
# Pick a random test sample
i = np.random.randint(0, len(X_test))
sample = X_test[i].reshape(1, -1)
true_label = y_test[i]
# Predict
pred_label = rf.predict(sample)[0]
print("True label:", true_label)
print("Predicted label:", pred_label)

True label: 0
Predicted label: 0
proba = rf.predict_proba(sample)[0]
print("Class probabilities:", proba)

Class probabilities: [0.9273452 0. 0. 0.0726548]
# Pick a random baseline window
baseline_sample = X_test[y_test == 0][0].reshape(1, -1)
stress_sample = X_test[y_test == 1][0].reshape(1, -1)
activity_sample = X_test[y_test == 2][0].reshape(1, -1)
print("Baseline →", rf.predict(baseline_sample)[0])
print("Stress →", rf.predict(stress_sample)[0])
print("Activity →", rf.predict(activity_sample)[0])

Baseline → 0
Stress → 1
Activity → 2
import numpy as np
```

```

import matplotlib.pyplot as plt
import pandas as pd

# Define feature names per sensor
ecg_cols = [f'ECG_{f}' for f in ['mean', 'std', 'max', 'min', 'hr_mean', 'hr_std', 'hrv_mean', 'hrv_std']]
eda_cols = [f'EDA_{f}' for f in ['mean', 'std', 'max', 'min', 'slope']]
acc_cols = [f'ACC_{f}' for f in ['x_mean', 'x_std', 'y_mean', 'y_std', 'z_mean', 'z_std', 'mag_mean', 'mag_std']]
temp_cols = [f'TEMP_{f}' for f in ['mean', 'std', 'max', 'min', 'slope']]

# Combine all
feature_names = ecg_cols + eda_cols + acc_cols + temp_cols

# Safety check
print("Total features:", len(feature_names))
print("Feature array shape:", final_features.shape)

# Feature importances
importances = rf.feature_importances_
indices = np.argsort(importances)[::-1][:15] # top 15

plt.figure(figsize=(10,5))
plt.barh(range(15), importances[indices][::-1], color='skyblue')
plt.yticks(range(15), [feature_names[i] for i in indices][::-1])
plt.xlabel("Importance")
plt.title("Top 15 Most Important Features")
plt.tight_layout()
plt.show()

Total features: 26
Feature array shape: (2378, 19)

```

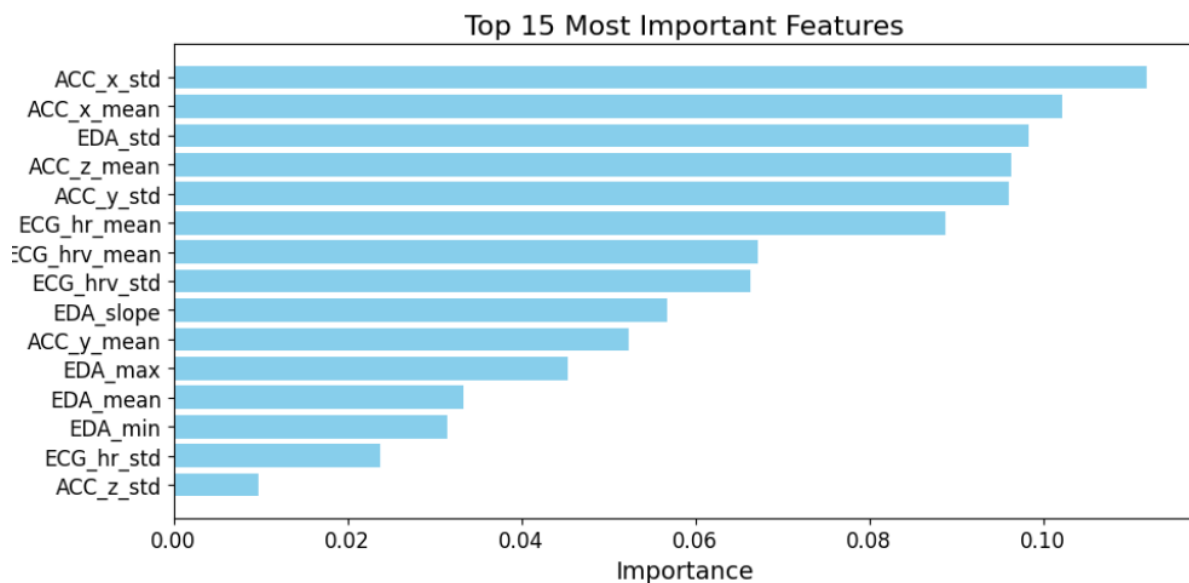


Fig 7.14 Top 15 most important features

References

1. Schmidt, P., Reiss, A., Duerichen, R., Marberger, C., & Van Laerhoven, K., “Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection,” Proceedings of the 20th ACM International Conference on Multimodal Interaction, 2018.
2. Gjoreski, M., Gjoreski, H., Luštrek, M., & Gams, M., “Continuous Stress Detection Using a Wrist Device: In Laboratory and Real Life,” Proceedings of the ACM International Joint Conference on Pervasive and Ubiquitous Computing, 2016.
3. Selye, H., “The Stress of Life,” McGraw-Hill Publication, New York, 1976.
4. Kurniawan, H., Maslov, A. V., & Pechenizkiy, M., “Stress Detection from Speech and Galvanic Skin Response Signals,” Proceedings of the International Symposium on Computer-Based Medical Systems, 2013.
5. Healey, J., & Picard, R., “Detecting Stress During Real-World Driving Tasks Using Physiological Sensors,” IEEE Transactions on Intelligent Transportation Systems, vol. 6, no. 2, pp. 156–166, 2005.
6. Breiman, L., “Random Forests,” Machine Learning Journal, vol. 45, no. 1, pp. 5–32, 2001.
7. Cortes, C., & Vapnik, V., “Support Vector Networks,” Machine Learning, vol. 20, pp. 273–297, 1995.
8. Goodfellow, I., Bengio, Y., & Courville, A., “Deep Learning,” MIT Press, 2016.
9. McMahan, B., Moore, E., Ramage, D., Hampson, S., & Aguera y Arcas, B., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” Proceedings of AISTATS, 2017.
10. Arduino Documentation, ESP32 and Sensor Interface Programming, Arduino IDE Documentation.
11. Scikit-learn Documentation, Machine Learning in Python, Available: <https://scikit-learn.org/>
12. TensorFlow Documentation, Deep Learning Framework Documentation, Available: <https://www.tensorflow.org/>
13. Python Software Foundation, Python Language Documentation, Available: <https://www.python.org/>
14. Kaggle WESAD Dataset Repository, Wearable Stress and Affect Detection Dataset.

Appendix

Appendix A: Hardware Components

ESP32 Microcontroller

- Dual-core microcontroller with Wi-Fi and Bluetooth support
- Used for sensor interfacing and wireless communication

MAX30100/MAX30102 Sensor

- Measures heart rate and blood oxygen levels
- Used for physiological signal acquisition

MPU6050 Motion Sensor

- Accelerometer and gyroscope sensor
- Used for activity and motion detection

Appendix B: Software Tools

Software	Purpose
Arduino IDE	Programming ESP32
Python	Data processing and ML implementation
Jupyter Notebook	Model development and testing
Google Colab	Cloud-based training environment

Appendix C: Dataset Information

WESAD Dataset

The WESAD (Wearable Stress and Affect Detection) dataset was used for training and evaluation of the proposed system.

Signals Used

- Electrodermal Activity (EDA)
- Blood Volume Pulse (BVP)
- Accelerometer (ACC)

Labels

- Baseline
- Stress
- Amusement

Appendix D: Machine Learning Workflow

1. Data Acquisition

2. Signal Preprocessing
3. Feature Extraction
4. Window Segmentation
5. Model Training
6. Model Evaluation
7. Federated Learning Integration

Appendix E: Evaluation Metrics

Accuracy

Measures the percentage of correctly classified samples.

$$\text{Accuracy} = (TP+TN) / (FP+FN + TP+TN)$$

Precision

Measures the proportion of correctly predicted positive samples.

$$\text{Precision} = \{TP\} / \{TP+FP\}$$

Recall

Measures the proportion of actual positive samples correctly identified.

$$\text{Recall} = (TP) / (TP+FN)$$

F1-Score

Harmonic mean of precision and recall.

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Appendix F: Sample Output Results

- Random Forest Accuracy: 97.68%
- Decision Tree Accuracy: 89.28%
- Neural Network Accuracy: 93%
- Logistic Regression Accuracy: 84%

The Random Forest classifier produced the best overall performance for stress detection using multimodal physiological signals.